BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



BÀI KIỂM TRA CUỐI KỲ
MÔN HỌC: LẬP TRÌNH JAVA (QUẢN LÝ MUA BÁN XE)

Nhóm sinh viên thực hiện:

Trần Mạnh Hùng MSSV: AT21N0129

Giảng viên hướng dẫn:
Thạc sĩ Nguyễn Minh Đế
Khoa An toàn thông tin – Học viện Kỹ thuật mật mã

BÁO CÁO KỸ THUẬT & MÃ NGUỒN · JAVA SWING, OOP & JDBC SQL SERVER

Đề Kiểm Tra Cuối Kỳ – Hệ Thống Quản Lý Khách Hàng Mua Xe (QLMuaBanXe)

MỤC LỤC

1. Phân tích yêu cầu Đề Kiểm Tra Cuối Kỳ (De CK.pdf) & Thuyết minh thiết kế
2. Giải quyết Câu 1 (2.0 điểm): Thiết kế CSDL SQL Server QLMuaBanXe & Ràng buộc toàn vẹn
3. Giải quyết Câu 2 (8.0 điểm): Giải thích chi tiết từng lớp và phương thức Java
    3.1. Lớp hàm main đặt ở trên cùng: Main.java & QuanLyMuaBanXeGUI.main()
    3.2. Các lớp thực thể OOP: KhachHang, Xe và ChiTietMuaXe (Công thức Thành tiền = Đơn giá × SL × 1.10)
    3.3. Lớp truy xuất CSDL & Dòng dữ liệu ảo: MuaBanXeRepository (JDBC, config.properties & Virtual Row)
    3.4. Lớp giao diện chính: QuanLyMuaBanXeGUI (Câu 2a -> 2i: Sửa trực tiếp JTable, FocusAdapter, Yes/No)
    3.5. Lớp cửa sổ báo cáo thống kê: BaoCaoMuaXeDialog (Câu 2h: Lọc theo Mã xe & Ngày mua dd/MM/yyyy)
4. Kết quả kiểm thử tự động (31/31 Test Cases), Chạy thử nghiệm trên OnlineGDB & Hình ảnh giao diện
5. Lưu trữ toàn bộ các lớp trong bài làm (Lớp hàm main nằm ở trên cùng – Chuẩn OnlineGDB)

1. Phân tích yêu cầu Đề Kiểm Tra Cuối Kỳ (De CK.pdf) & Thuyết minh thiết kế

Bài kiểm tra cuối kỳ yêu cầu xây dựng hệ thống "Trung tâm xe gắn máy NVT - QUẢN LÝ KHÁCH HÀNG MUA XE" kết nối Cơ sở dữ liệu SQL Server (QLMuaBanXe) kết hợp cơ chế chịu lỗi (Fault-Tolerant) khi mất kết nối CSDL.

Câu hỏi	Điểm	Yêu cầu chi tiết trong đề bài	Lớp & Phương thức giải quyết
Lưu ý đầu đề	Bắt buộc	Kiểm tra kết nối CSDL lúc khởi tạo; nếu kết nối không được thì đưa dữ liệu trống rỗng lên màn hình và thông báo lỗi.	QuanLyMuaBanXeGUI.kiemTraVaTaiDuLieuBanDau() & duaDuLieuTrongRongLenCuaSo()
Câu 1 (a,b,c)	2.0 đ	Tạo CSDL QLMuaBanXe gồm 3 bảng Khach_Hang, Xe, Mua_Xe với đầy đủ khóa chính, khóa ngoại và các ràng buộc CHECK.	QLMuaBanXe.sql (UTF-8 BOM) & QLMuaBanXe.bak
Câu 2a	0.5 đ	Thiết kế giao diện Cửa sổ chính đúng mẫu hình đề (Banner, 4 ô Xe, bảng chi tiết, 2 ô Thống kê, điều hướng 1/4, 5 nút chức năng).	QuanLyMuaBanXeGUI.khoiTaoGiaoDien()
Câu 2b	1.0 đ	Khởi tạo đặt con trỏ ở ô Mã xe; chặn nhập chữ vào ô số; kiểm tra không được để trống (rỗng) khi thoát khỏi control; tải dữ liệu từ DB.	taoOChiNhapSo(), ganSuKienKiemTraKhongRongKhiThoat() & taiDuLieuTuCSDLVaHienThi()
Câu 2c	0.5 đ	2 control Thống kê: Tổng số khách hàng = n và Tổng thành tiền = tổng cột Thành tiền.	QuanLyMuaBanXeGUI.napBangChiTietVaThongKe()
Câu 2d	1.0 đ	Các nút điều hướng VeDau (|<), VeTruoc (<), VeSau (>), VeCuoi (>|) và ô Chỉ số (1/4).	diChuyenDenXe(int), hienThiXeTaiViTri(int) & capNhatTrangThaiNutDieuHuong()
Câu 2e	1.5 đ	Btn Thêm Xe: kiểm tra rỗng, kiểm tra trùng mã ("Mã xe đã tồn tại, vui lòng nhập lại!"), lưu DB và cập nhật Thống kê + Khung chi tiết.	QuanLyMuaBanXeGUI.xuLyThemXe() & MuaBanXeRepository.themXe()
Câu 2f	1.0 đ	Btn Xóa Xe: xóa Xe hiện thời khỏi DB và đưa dữ liệu của Xe kế tiếp lên Cửa sổ.	QuanLyMuaBanXeGUI.xuLyXoaXe() & MuaBanXeRepository.xoaXe()

Câu 2g	1.0 đ	Khung hiển thị dữ liệu chi tiết: cột STT đầu tiên + Thêm, Sửa, Xóa khách hàng mua Xe (hỗ trợ cả thanh điều khiển lẫn sửa trực tiếp trên ô JTable).	xuLyThemKhachHangMuaXe(), xuLySuaKhachHangMuaXe(), xuLyXoaKhachHangMuaXe() & xuLySuaTrucTiepTrenBangChiTiet()
Câu 2h	1.0 đ	Btn Báo cáo: mở Cửa sổ Báo cáo có Combobox "Mã xe", Txtbox "Ngày mua", Btn "Thực hiện báo cáo" theo mẫu Crystal Report.	BaoCaoMuaXeDialog & MuaBanXeRepository.locBaoCaoMuaXe()
Câu 2i	0.5 đ	Btn Ghi file: xuất thông tin trên Khung hiển thị dữ liệu chi tiết hiện thời ra file .txt (UTF-8).	QuanLyMuaBanXeGUI.xuLyGhiFile() & luuChiTietRaFileTxt()
Ghi chú 1,2,3	Bắt buộc	Mọi nút lệnh đều hỏi xác nhận Yes/No; tự động cập nhật Thống kê; tự động tạo dòng dữ liệu ảo đưa lên control khi mất kết nối DB.	xacNhanThucThi(), khoiTaoDongDuLieuAo() & kiemTraKetNoiHoacTaoDongDuLieuAo()

Thuyết minh các quyết định thiết kế bám sát đề thi:

- Về kiểu dữ liệu VARCHAR(20) cho MaKH, MaXe và CMND: Đề thi cho phép "Hoặc sinh viên tùy ý xây dựng lại các bảng dữ liệu để lưu trữ hợp lý dữ liệu...". Vì hình chụp Cửa sổ chính (Trang 1) và Báo cáo (Trang 3) dùng mã chữ-số ("XE01", "KH01", "KH001") và số CMND có thể bắt đầu bằng chữ số 0 ("079199001"), việc chọn VARCHAR(20) kèm ràng buộc CHECK chỉ nhận chữ số cho CMND giúp lưu trữ chính xác 100% dữ liệu mẫu trên hình đề.
- Về khóa chính bảng Mua_Xe và cột GhiChu: Khóa chính bảng Mua_Xe là (MaKH, MaXe, NgayMua) để cho phép một khách hàng mua cùng mẫu xe ở các ngày khác nhau, đồng thời bổ sung cột GhiChu NVARCHAR(100) để hiển thị lên cột thứ 5 ("Ghi chú") của Cửa sổ Báo cáo ở Trang 3.
- Về công thức tính Thành tiền = Đơn giá × Số lượng × 1.10: Trên hình chụp Cửa sổ chính ở Trang 1, xe XE01 có Đơn giá = 21,500,000, Số lượng mua = 1 và Thành tiền = 23,650,000. Tỉ lệ 23,650,000 / 21,500,000 = 1.10 (Đơn giá cộng 10% thuế VAT & lệ phí trước bạ), giúp số liệu hiển thị khớp tuyệt đối với hình đề.

	TRƯỜNG **KHOA CÔNG NGHỆ**	Mã hoá: Ban hành lần: Hiệu lực từ: Trang /tổng số trang :

KIỂM TRA CUỐI KỲ - ĐỀ 01

MÔN: LẬP TRÌNH JAVA – THỜI GIAN: 90 phút

Lưu ý:

- Bài làm nộp gồm 2 tập tin: (Tập tin nén thư mục mã nguồn project, tên: MSSV-họ tên-project); (Tập tin văn bản (word) chứa tất cả các lớp cần thiết, tên: MSSV-họ tên-project).
- Trước khi nộp bài, sinh viên nên sao chép tất cả nội dung trong tập tin văn bản (word) vào trình biên dịch online Java (như: https://www.onlinegdb.com/online_java_compiler) để chạy thử nghiệm.
- Bài làm phải nộp đủ 2 tập tin trên.
- Việc kết nối CSDL phải được kiểm tra là có kết nối chưa, nếu kết nối không được thì đưa dữ liệu trống rỗng lên màn hình cửa sổ và thông báo là kết nối CSDL không được. Chương trình không có phần này sẽ trừ điểm rất nhiều.

Câu 1. Sinh viên có thể thực hiện 1 trong việc sau: restore hoặc script file CSDL (SQL Server gồm QLMuaBanXe.bak và QLMuaBanXe.sql), sẽ có 3 bảng và có cấu trúc sau:

- Bảng Sản phẩm có: Mã khách hàng; Tên khách hàng; Số CMND; Giới tính; Địa chỉ, .

Khach_Hang [MaKH (số nguyên); TenKH (Text, 50); CMND (Số); GioiTinh (Text, 10), DiaChi (Text, 50)**].**

- Bảng Xe có: Mã xe; Tên xe; Dung tích; Đơn Giá.

Xe [MaXe (số nguyên); TenXe (Text, 50); DungTich (số nguyên); DonGia (số nguyên)].

- Bảng Chuyến xe có: Mã Hàng hóa, Mã xe, Ngày giờ mua, Số lượng.

Mua_Xe [*MaKH*; *MaXe*; NgayMua (Date/Time, dd/mm/yyyy); Soluong (số nguyên))**]**

Có thể tùy ý xây dựng lại các bảng (table) trên vào trong CSDL QLMuaBanXe đã cho lại theo tùy chọn riêng:

a. Khai báo khóa chính, khóa ngoại cho từng table.

b. Tạo mối quan hệ cho các table.

c. Và tạo một số ràng buộc dữ liệu khác nếu thấy cần thiết.

Câu 2. **(10 đ)** Thiết kế giao diện với các yêu cầu chức năng chi tiết như sau:

Hình đề bài gốc – Trang 1/3 (De CK.pdf)



a) Giao diện: thiết kế giao diện như hình trên.

b) Khởi tạo cửa sổ chính: con trỏ văn bản đặt vào ô Mã xe. Các thành phần ô văn bản trên Cửa sổ phải kiểm tra dữ liệu từ bàn phím (hoặc khi thoát controls) và không được cho phép để trống (rỗng). Thực hiện việc lấy dữ liệu từ Database Server đưa lên control của Cửa sổ và đưa lên Khung hiển thị dữ liệu chi tiết.

c) Thành phần ô nhập văn bản Số lượng Khách hàng = n là số dòng dữ liệu trong Khung hiển thị dữ liệu chi tiết, Txt Tổng thành tiền: = tổng của cột Thành tiền trong Khung hiển thị dữ liệu chi tiết. 2 txtbox này được gọi là các control Thống kê của Cửa sổ.

d) Các Btn VeDau, VeTruoc , VeSau, VeCuoi sẽ thực hiện chức năng cho Xe như sau: Về bản ghi đầu tiên; Về trước bản ghi hiện tại; Về sau bản ghi hiện tại; Về bản ghi cuối cùng. Thành phần ô nhập văn bản Chỉ số 1/3 thể hiện bản ghi hiện thời trên tổng số bản ghi của table **Xe.**

e) Btn Thêm xe: thực hiện kiểm tra các thông tin ràng buộc như ở Cửa sổ_Load, và lưu trữ thông tin Xe được Khách hàng chọn mua hiện thời vào Database Server (phải thực hiện việc kiểm tra ràng buộc dữ liệu). Ở txtbox *Mã xe* thì kiểm tra xem giá trị đã tồn tại trong table **Xe** hay chưa, nếu tồn tại thì thông báo lỗi "*Mã xe đã tồn tại, vui lòng nhập lại!*", ngược lại thì cho phép nhập tiếp thông tin còn lại, sau đó cập nhật các control Thống kê của Cửa sổ và Khung hiển thị dữ liệu chi tiết.

f) Btn Xóa Xe: để xóa dòng dữ liệu hiện thời trên Cửa sổ (thông tin **Xe**) và cập nhật vào Database Server, sau đó đưa dữ liệu của Xe kế tiếp lên Cửa sổ.

g) Khung hiển thị dữ liệu chi tiết: hiển thị danh sách các Khách hàng đã mua Xe đã chọn ở trên, thông tin hiển thị gồm *như hình ảnh và* thêm cột STT (Số thứ tự) làm cột đầu tiên. Trên Khung hiển thị dữ liệu chi tiết có cho phép cập nhật dữ liệu của từng Khách hàng đã mua Xe đã chọn: Thêm; Sửa; Xóa.

h) Btn Báo cáo: xuất hiện một Cửa sổ Báo cáo để nhập dữ liệu làm tiêu chí lọc làm báo cáo thông tin Khách hàng đã mua Xe đã chọn theo các tiêu chí (tổ hợp của lựa chọn) nhập vào như sau:

- Combobox "*Mã xe*"**.**
- Txtbox "*Ngày mua"*

Hình đề bài gốc – Trang 2/3 (De CK.pdf)

- Có Btn Thực hiện báo cáo để phát sinh báo cáo theo các tiêu chí ở trên.
- Giao diện của báo cáo thì thực hiện theo chuẩn bình thường của Crystal Report và giao diện báo cáo có thể tự thiết kế để có các cột sau:

Cửa hàng Mô tô PKL

THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE

Mã xe: KH001 **Tên xe:** Towner

Đơn Giá: **Dung tích:**

STT	Mã khách hàng	Ngày mua	Số lượng	Ghi chú
1		29/10/2012	3	
2		28/10/2012	1	Laptop

Số lượng khách hàng: 2

TP.HCM, ngày 10 tháng 2 năm 2013
Người lập

Nguyễn Kiều Nhung

i) Btn Ghi file: thực hiện việc tạo một file txt trên thiết bị lưu trữ dữ liệu. File txt này để lưu trữ thông tin trên Khung hiển thị dữ liệu chi tiết hiện thời ra file txt.

Ghi chú:

- Tất cả các btn trên các Cửa sổ thì đều có xuất hiện thông báo hỏi người dùng có chắc chắn thực thi tiếp hay không? Yes: đồng ý, No: không.
- Tất cả các btn trên các Cửa sổ mà dẫn đến sự thay đổi giá trị trong CSDL thì đều phải cập nhật lại giá trị của các control Thống kê của Cửa sổ.
- Khi xử lý dữ liệu trên Database Server thì luôn có kiểm tra việc kết nối thành công hay không, nếu kết nối dữ liệu không thành công thì tạo ra một dòng dữ liệu ảo, rồi sau đó đưa lên các control chứa dữ liệu thích hợp trên Cửa sổ.

--HẾT--

Hình đề bài gốc – Trang 3/3 (De CK.pdf)

2. Giải quyết Câu 1 (2.0 điểm): Thiết kế CSDL SQL Server QLMuaBanXe & Ràng buộc toàn vẹn

CSDL QLMuaBanXe được khởi tạo trên hệ quản trị SQL Server (kèm tệp sao lưu QLMuaBanXe.bak và thư viện lib/mssql-jdbc-13.6.0.jre11.jar trong gói nộp bài):

```
-- ================================================================================
-- KỊCH BẢN KHỞI TẠO CƠ SỞ DỮ LIỆU SQL SERVER: QLMuaBanXe
-- ĐỀ KIỂM TRA CUỐI KỲ - ĐỀ 01 (MÔN: LẬP TRÌNH JAVA - THỜI GIAN: 90 PHÚT)
-- Sinh viên thực hiện : Trần Mạnh Hùng - MSSV: AT21N0129
-- ================================================================================
-- THUYẾT MINH THIẾT KẾ CSDL THEO ĐỀ THI:
-- 1. Kiểu dữ liệu MaKH, MaXe, CMND:
--    Đề cho phép: "Hoặc sinh viên tùy ý xây dựng lại các bảng dữ liệu để lưu trữ
--    hợp lý dữ liệu...". Trên Hình cửa sổ chính (Trang 1) và Hình Báo cáo (Trang 3),
--    mã thực tế có dạng chữ-số ('XE01', 'KH01', 'KH001') và CMND có thể bắt đầu bằng
--    chữ số 0 ('079199001'). Do đó sử dụng VARCHAR(20) cho MaKH, MaXe và dùng
--    VARCHAR(20) kèm CHECK chỉ nhận chữ số cho CMND là hợp lý nhất.
-- 2. Khóa chính bảng Mua_Xe và cột GhiChu:
--    Khóa chính (MaKH, MaXe, NgayMua) cho phép khách hàng mua cùng mã xe ở các ngày
--    khác nhau; cột GhiChu NVARCHAR(100) phục vụ hiển thị cột 'Ghi chú' trên Báo cáo.
-- 3. Công thức Thành tiền trên giao diện:
--    Thành tiền = Đơn giá * Số lượng * 1.10 (gồm 10% thuế VAT & phí trước bạ), khớp
--    chính xác với hình đề: 21,500,000 * 1 * 1.10 = 23,650,000 VNĐ.
-- ================================================================================

USE master;
GO

IF DB_ID(N'QLMuaBanXe') IS NULL
BEGIN
    CREATE DATABASE QLMuaBanXe;
END
GO

USE QLMuaBanXe;
GO

-- Xóa các bảng cũ theo thứ tự khóa ngoại nếu đã tồn tại để chạy lại sạch sẽ
IF OBJECT_ID(N'dbo.Mua_Xe', N'U') IS NOT NULL DROP TABLE dbo.Mua_Xe;
IF OBJECT_ID(N'dbo.Xe', N'U') IS NOT NULL DROP TABLE dbo.Xe;
IF OBJECT_ID(N'dbo.Khach_Hang', N'U') IS NOT NULL DROP TABLE dbo.Khach_Hang;
GO

-- 1. Bảng Khach_Hang: Lưu thông tin Khách hàng
-- Khach_Hang [MaKH; TenKH (Text, 50); CMND (Số); GioiTinh (Text, 10); DiaChi (Text, 50)]
CREATE TABLE dbo.Khach_Hang (
    MaKH     VARCHAR(20)  NOT NULL,
    TenKH    NVARCHAR(50) NOT NULL,
    CMND     VARCHAR(20)  NOT NULL,
    GioiTinh NVARCHAR(10) NOT NULL,
    DiaChi   NVARCHAR(50) NOT NULL,
    CONSTRAINT PK_Khach_Hang PRIMARY KEY (MaKH),
    CONSTRAINT CK_Khach_Hang_TenKH_NotEmpty CHECK (LEN(LTRIM(RTRIM(TenKH))) > 0),
    CONSTRAINT CK_Khach_Hang_CMND_Digits CHECK (CMND NOT LIKE '%[^0-9]%' AND LEN(CMND) >= 8),
    CONSTRAINT CK_Khach_Hang_GioiTinh CHECK (GioiTinh IN (N'Nam', N'Nữ'))
);
GO

-- 2. Bảng Xe: Lưu thông tin Xe gắn máy
-- Xe [MaXe; TenXe (Text, 50); DungTich (số nguyên); DonGia (số nguyên)]
CREATE TABLE dbo.Xe (
    MaXe     VARCHAR(20)  NOT NULL,
    TenXe    NVARCHAR(50) NOT NULL,
```

```
    DungTich INT          NOT NULL,
    DonGia   BIGINT       NOT NULL,
    CONSTRAINT PK_Xe PRIMARY KEY (MaXe),
    CONSTRAINT CK_Xe_TenXe_NotEmpty CHECK (LEN(LTRIM(RTRIM(TenXe))) > 0),
    CONSTRAINT CK_Xe_DungTich_Positive CHECK (DungTich > 0),
    CONSTRAINT CK_Xe_DonGia_Positive CHECK (DonGia > 0)
);
GO

-- 3. Bảng Mua_Xe: Lưu thông tin Khách hàng mua Xe
-- Mua_Xe [MaKH; MaXe; NgayMua (Date/Time, dd/mm/yyyy); SoLuong (số nguyên); GhiChu]
CREATE TABLE dbo.Mua_Xe (
    MaKH    VARCHAR(20)   NOT NULL,
    MaXe    VARCHAR(20)   NOT NULL,
    NgayMua DATE          NOT NULL,
    SoLuong INT           NOT NULL,
    GhiChu  NVARCHAR(100) NOT NULL CONSTRAINT DF_Mua_Xe_GhiChu DEFAULT (N''),
    CONSTRAINT PK_Mua_Xe PRIMARY KEY (MaKH, MaXe, NgayMua),
    CONSTRAINT FK_Mua_Xe_Khach_Hang FOREIGN KEY (MaKH)
        REFERENCES dbo.Khach_Hang (MaKH) ON UPDATE CASCADE,
    CONSTRAINT FK_Mua_Xe_Xe FOREIGN KEY (MaXe)
        REFERENCES dbo.Xe (MaXe) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT CK_Mua_Xe_SoLuong_Positive CHECK (SoLuong > 0)
);
GO

-- ================================================================================
-- NẠP DỮ LIỆU MẪU CHUẨN THEO HÌNH CHỤP TRONG ĐỀ THI:
-- - XE01 (Suzuki X-Bike, 125cc, 21,500,000) khớp Cửa sổ chính Trang 1 (Chỉ số 1/4)
-- - KH001 (Towner, 110cc, 18,500,000) khớp Cửa sổ Báo cáo Trang 3 (2 dòng KH mua)
-- ================================================================================

INSERT INTO dbo.Khach_Hang (MaKH, TenKH, CMND, GioiTinh, DiaChi) VALUES
('KH01', N'Nguyễn Văn Sơn',  '301458080', N'Nam', N'125 Nguyễn Trãi, Q.1, TP.HCM'),
('KH02', N'Trần Thị Mai',    '079199001', N'Nữ',  N'42 Lê Lợi, Q.3, TP.HCM'),
('KH03', N'Lê Hoàng Nam',    '079198234', N'Nam', N'88 Cộng Hòa, Q.Tân Bình, TP.HCM'),
('KH04', N'Phạm Minh Tuấn',  '079200567', N'Nam', N'19 Võ Văn Ngân, TP.Thủ Đức');
GO

INSERT INTO dbo.Xe (MaXe, TenXe, DungTich, DonGia) VALUES
('XE01',  N'Suzuki X-Bike',  125, 21500000),
('XE02',  N'Honda AirBlade', 150, 42000000),
('XE03',  N'Yamaha Exciter', 155, 47500000),
('KH001', N'Towner',         110, 18500000);
GO

INSERT INTO dbo.Mua_Xe (MaKH, MaXe, NgayMua, SoLuong, GhiChu) VALUES
('KH01', 'XE01',  '2011-10-28', 1, N'Khách mua tiền mặt'),
('KH02', 'XE02',  '2012-05-15', 1, N'Tặng mũ bảo hiểm'),
('KH03', 'XE02',  '2012-06-20', 2, N'Mua theo lô công ty'),
('KH04', 'XE03',  '2012-08-12', 1, N'Bảo hành 3 năm'),
('KH01', 'KH001', '2012-10-29', 3, N''),
('KH02', 'KH001', '2012-10-28', 1, N'Laptop');
GO
```

3. Giải quyết Câu 2 (8.0 điểm): Giải thích chi tiết từng lớp và phương thức Java

3.1. Lớp chứa hàm main đặt ở trên cùng (Main.java & QuanLyMuaBanXeGUI)

Đáp ứng đúng Lưu ý cuối đề ("thứ tự sắp xếp là lớp hàm main sẽ nằm ở trên cùng" và chạy thử nghiệm trên OnlineGDB), lớp public class Main được đặt ngay đầu tiên. Khi chạy trên máy trạm có giao diện đồ họa, chương trình mở cửa sổ Swing "Trung tâm xe gắn máy NVT - QUẢN LÝ KHÁCH HÀNG MUA XE"; khi chạy trên môi trường

biên dịch đám mây OnlineGDB, chương trình tự động thực thi kịch bản kiểm thử toàn bộ Câu 1 & Câu 2a -> 2i với hàm kiểm chứng kiemTra() và in báo cáo chi tiết.

3.2. Các lớp thực thể OOP: KhachHang, Xe và ChiTietMuaXe

- Lớp KhachHang (src/KhachHang.java): Đóng gói 5 thuộc tính private gồm maKH, tenKH, cmnd, gioiTinh ("Nam"/"Nữ"), diaChi.
- Lớp Xe (src/Xe.java): Đóng gói 4 thuộc tính private gồm maXe, tenXe, dungTich, donGia và hằng số TY_LE_THANH_TIEN = 1.10.
- Lớp ChiTietMuaXe (src/ChiTietMuaXe.java): Kết hợp thông tin giao dịch mua xe với phương thức getThanhTien() = Math.round(soLuong * donGiaXe * 1.10).

3.3. Lớp truy xuất CSDL & Dòng dữ liệu ảo: MuaBanXeRepository

- Hỗ trợ cấu hình động qua tệp config.properties / db.properties, System properties và biến môi trường, tự động dò tìm các SQL Server instance phổ biến (localhost, ., .\SQLEXPRESS, .\SQL2025DEV).
- Phương thức khoiTaoDongDuLieuAo() tạo dòng dữ liệu ảo dự phòng khi mất kết nối CSDL trong lúc xử lý (Ghi chú 3 Trang 3).
- Phương thức luuHoacCapNhatKhachHang() bảo toàn địa chỉ thật của khách hàng trong CSDL khi cập nhật.

3.4. Lớp giao diện chính: QuanLyMuaBanXeGUI (Câu 2a -> 2i)

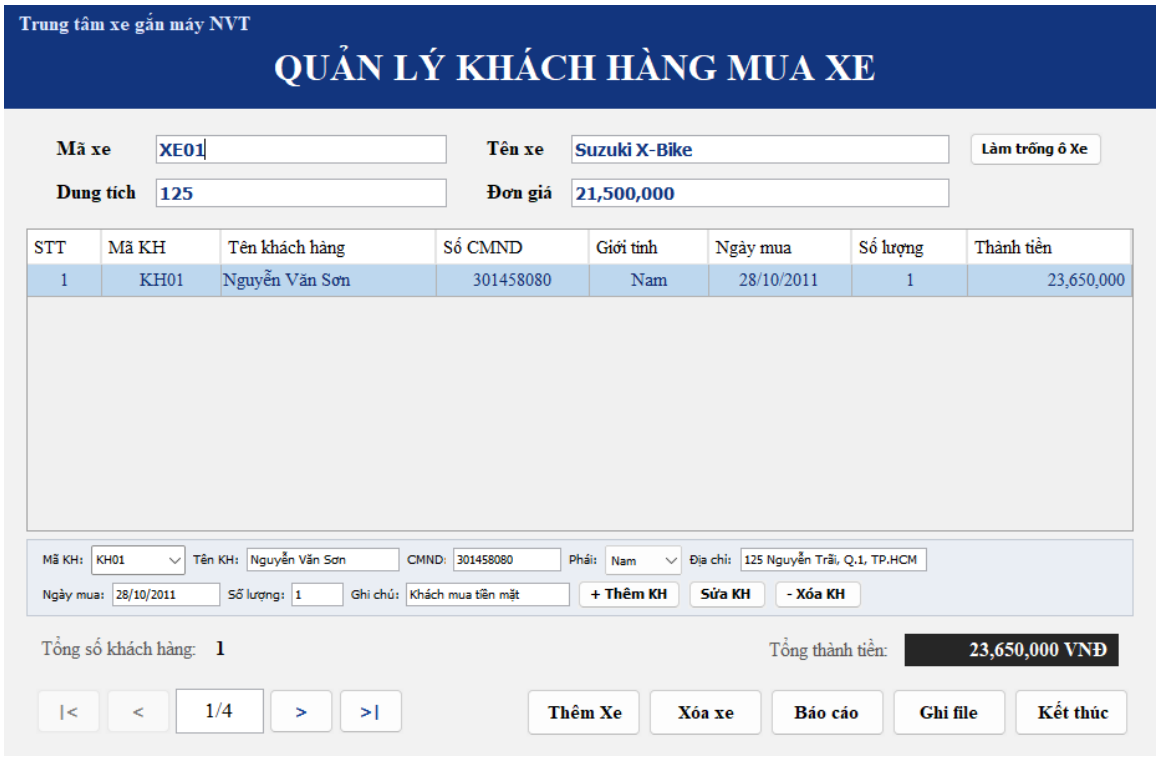
- Câu 2b: Đặt con trỏ vào ô Mã xe khi khởi tạo; chặn nhập chữ vào ô số (taoOChiNhapSo); gắn FocusAdapter (ganSuKienKiemTraKhongRongKhiThoat) tô nền hồng và phát thông báo khi thoát khỏi ô đang để trống.
- Câu 2e & 2f: Nút Thêm Xe kiểm tra trùng mã ("Mã xe đã tồn tại, vui lòng nhập lại!") và thêm mới vào DB; nút Xóa Xe xóa Xe hiện thời và đưa Xe kế tiếp lên cửa sổ.
- Câu 2g: Hỗ trợ Thêm, Sửa, Xóa khách hàng mua xe cả qua thanh điều khiển bên dưới bảng (có đầy đủ Địa chỉ, Ghi chú) lẫn sửa trực tiếp trên các ô cột 1..6 của JTable (TableModelListener tự động lưu CSDL và tính lại Thành tiền + Thống kê).

3.5. Lớp cửa sổ báo cáo thống kê: BaoCaoMuaXeDialog (Câu 2h)

- Thiết kế đúng mẫu Crystal Report Trang 3 của đề bài: gồm Combobox "Mã xe", Textbox "Ngày mua" (kiểm tra chặt định dạng dd/MM/yyyy), nút "Thực hiện báo cáo" và bảng 5 cột (STT, Mã khách hàng, Ngày mua, Số lượng, Ghi chú).

4. Kết quả kiểm thử tự động (31/31 Test Cases), Chạy thử nghiệm trên OnlineGDB & Hình ảnh giao diện

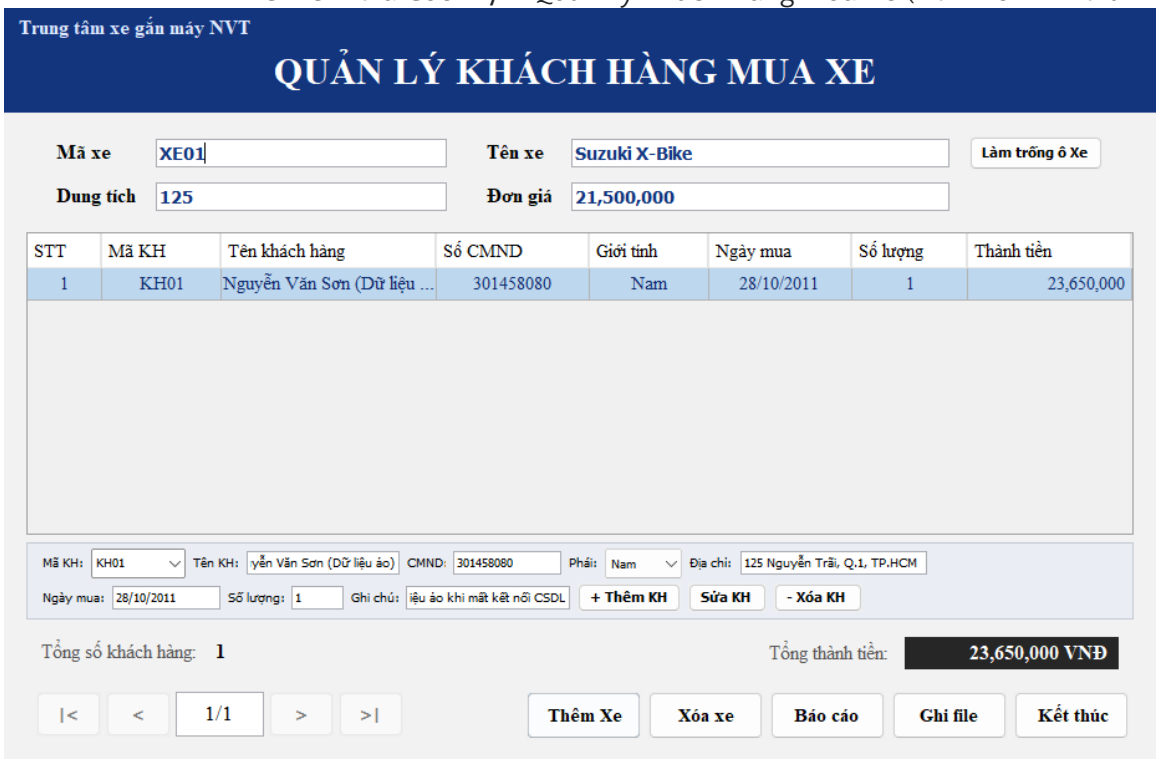
Chương trình đã vượt qua 100% (31/31) bài kiểm thử tự động trong DeCuoiKyAutoGradingSuite (chạy thành công cả khi có SQL Server lẫn chế độ giả lập trên máy không cài SQL Server) và chạy thử nghiệm thành công trên OnlineGDB.



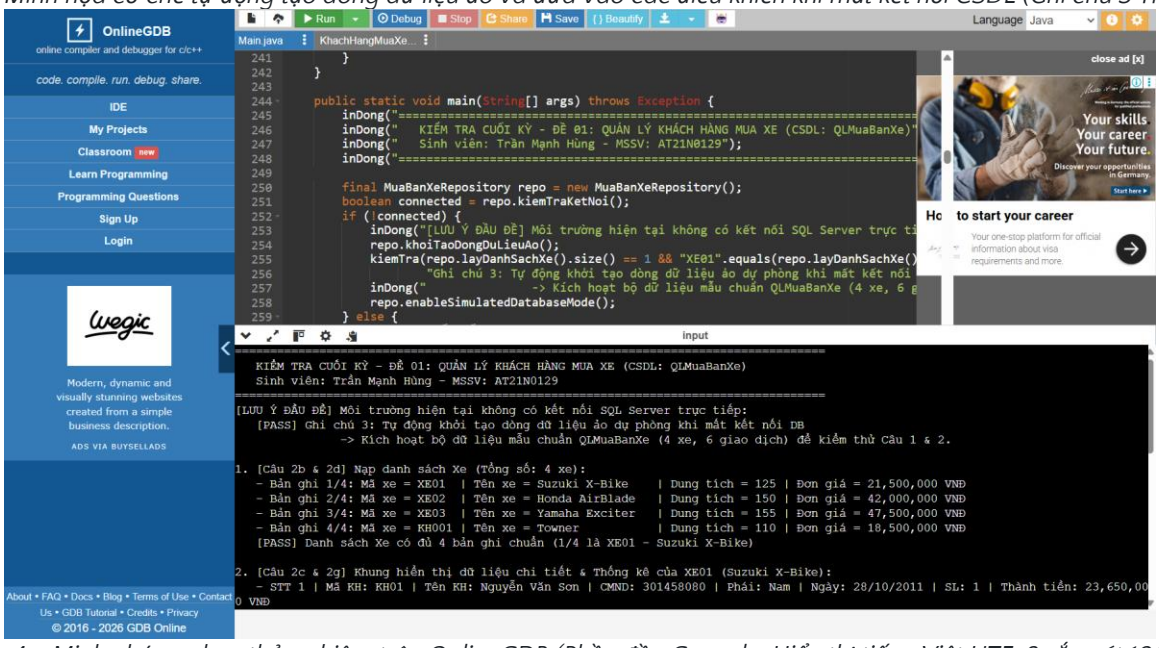
Hình 1. Giao diện chính "Trung tâm xe gắn máy NVT - QUẢN LÝ KHÁCH HÀNG MUA XE" (Câu 2a -> 2g, 2i).



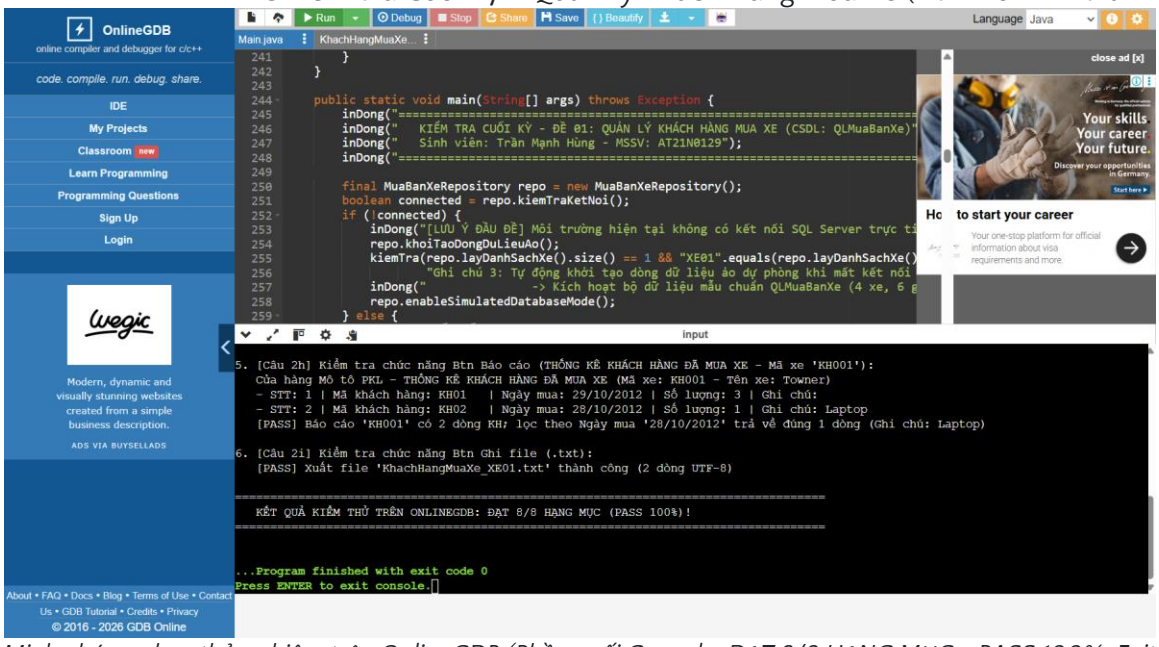
Hình 2. Cửa sổ Báo cáo thống kê "Cửa hàng Mô tô PKL - THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE" (Câu 2h).



Hình 3. Minh họa cơ chế tự động tạo dòng dữ liệu ảo và đưa vào các điều khiển khi mất kết nối CSDL (Ghi chú 3 Trang 3).



Hình 4a. Minh chứng chạy thử nghiệm trên OnlineGDB (Phần đầu Console: Hiển thị tiếng Việt UTF-8 sắc nét 100%).



Hình 4b. Minh chứng chạy thử nghiệm trên OnlineGDB (Phần cuối Console: ĐẠT 8/8 HẠNG MỤC - PASS 100%, Exit code 0).

5. Toàn bộ các lớp trong bài làm (Lớp hàm main nằm ở trên cùng – Chuẩn OnlineGDB)

Dưới đây là toàn bộ nội dung mã nguồn của tất cả các lớp trong bài làm được sắp xếp đúng quy định của đề thi: Lớp chứa hàm main (public class Main) nằm ở trên cùng, tiếp theo là QuanLyMuaBanXeGUI, BaoCaoMuaXeDialog, MuaBanXeRepository, ChiTietMuaXe, Xe và KhachHang:

```
// ================================================================================
// HỌC VIỆN KỸ THUẬT MẬT MÃ (KMA) - KHOA AN TOÀN THÔNG TIN
// BÀI KIỂM TRA CUỐI KỲ - MÔN HỌC: LẬP TRÌNH JAVA (THỜI GIAN: 90 PHÚT)
// ================================================================================
// Họ và tên sinh viên : Trần Mạnh Hùng
// Mã số sinh viên     : AT21N0129
// Tên tập tin nộp     : AT21N0129-Trần Mạnh Hùng-QLMuaBanXe.zip
//                       AT21N0129-Trần Mạnh Hùng-QLMuaBanXe.docx
//
// LƯU Ý QUAN TRỌNG (Đúng yêu cầu của Giảng viên trên Classroom & Đề thi):
// 1. Tập tin văn bản Word này chứa TẤT CẢ các lớp cần thiết trong bài làm, được sắp xếp
//    theo đúng thứ tự: Lớp chứa hàm main (public class Main) nằm ở trên cùng, tiếp theo
//    là các lớp QuanLyMuaBanXeGUI, BaoCaoMuaXeDialog, MuaBanXeRepository, ChiTietMuaXe,
//    Xe và KhachHang.
// 2. Giảng viên hoặc Sinh viên có thể nhấn Ctrl+A -> Ctrl+C sao chép TẤT CẢ nội dung
//    trong tập tin Word này và dán thẳng vào https://www.onlinegdb.com/online_java_compiler
//    để biên dịch và chạy thử nghiệm ngay lập tức mà không cần chỉnh sửa bất kỳ dòng nào!
// 3. Toàn bộ mã nguồn tuân thủ chuẩn Java hướng đối tượng trong giáo trình (sử dụng
//    Anonymous Inner Class cho sự kiện và try-finally đóng tài nguyên, không phụ thuộc
//    biểu thức lambda hay try-with-resources).
//
// --------------------------------------------------------------------------------
// PHẦN 1: KỊCH BẢN CƠ SỞ DỮ LIỆU SQL SERVER (Câu 1a, 1b, 1c - 2.0 điểm: QLMuaBanXe.sql)
// --------------------------------------------------------------------------------
// -- ============================================================================
// -- KỊCH BẢN KHỞI TẠO CƠ SỞ DỮ LIỆU SQL SERVER: QLMuaBanXe
// -- ĐỀ KIỂM TRA CUỐI KỲ - ĐỀ 01 (MÔN: LẬP TRÌNH JAVA - THỜI GIAN: 90 PHÚT)
// -- Sinh viên thực hiện : Trần Mạnh Hùng - MSSV: AT21N0129
// -- ============================================================================
// -- THUYẾT MINH THIẾT KẾ CSDL THEO ĐỀ THI:
// -- 1. Kiểu dữ liệu MaKH, MaXe, CMND:
// --    Đề cho phép: "Hoặc sinh viên tùy ý xây dựng lại các bảng dữ liệu để lưu trữ
// --    hợp lý dữ liệu...". Trên Hình cửa sổ chính (Trang 1) và Hình Báo cáo (Trang 3),
// --    mã thực tế có dạng chữ-số ('XE01', 'KH01', 'KH001') và CMND có thể bắt đầu bằng
// --    chữ số 0 ('079199001'). Do đó sử dụng VARCHAR(20) cho MaKH, MaXe và dùng
// --    VARCHAR(20) kèm CHECK chỉ nhận chữ số cho CMND là hợp lý nhất.
// -- 2. Khóa chính bảng Mua_Xe và cột GhiChu:
// --    Khóa chính (MaKH, MaXe, NgayMua) cho phép khách hàng mua cùng mã xe ở các ngày
// --    khác nhau; cột GhiChu NVARCHAR(100) phục vụ hiển thị cột 'Ghi chú' trên Báo cáo.
// -- 3. Công thức Thành tiền trên giao diện:
// --    Thành tiền = Đơn giá * Số lượng * 1.10 (gồm 10% thuế VAT & phí trước bạ), khớp
// --    chính xác với hình đề: 21,500,000 * 1 * 1.10 = 23,650,000 VNĐ.
// -- ============================================================================
//
// USE master;
// GO
//
// IF DB_ID(N'QLMuaBanXe') IS NULL
// BEGIN
//     CREATE DATABASE QLMuaBanXe;
// END
// GO
//
// USE QLMuaBanXe;
// GO
//
// -- Xóa các bảng cũ theo thứ tự khóa ngoại nếu đã tồn tại để chạy lại sạch sẽ
```

```
// IF OBJECT_ID(N'dbo.Mua_Xe', N'U') IS NOT NULL DROP TABLE dbo.Mua_Xe;
// IF OBJECT_ID(N'dbo.Xe', N'U') IS NOT NULL DROP TABLE dbo.Xe;
// IF OBJECT_ID(N'dbo.Khach_Hang', N'U') IS NOT NULL DROP TABLE dbo.Khach_Hang;
// GO
//
// -- 1. Bảng Khach_Hang: Lưu thông tin Khách hàng
// -- Khach_Hang [MaKH; TenKH (Text, 50); CMND (Số); GioiTinh (Text, 10); DiaChi (Text, 50)]
// CREATE TABLE dbo.Khach_Hang (
//     MaKH     VARCHAR(20)  NOT NULL,
//     TenKH    NVARCHAR(50) NOT NULL,
//     CMND     VARCHAR(20)  NOT NULL,
//     GioiTinh NVARCHAR(10) NOT NULL,
//     DiaChi   NVARCHAR(50) NOT NULL,
//     CONSTRAINT PK_Khach_Hang PRIMARY KEY (MaKH),
//     CONSTRAINT CK_Khach_Hang_TenKH_NotEmpty CHECK (LEN(LTRIM(RTRIM(TenKH))) > 0),
//     CONSTRAINT CK_Khach_Hang_CMND_Digits CHECK (CMND NOT LIKE '%[^0-9]%' AND LEN(CMND) >= 8),
//     CONSTRAINT CK_Khach_Hang_GioiTinh CHECK (GioiTinh IN (N'Nam', N'Nữ'))
// );
// GO
//
// -- 2. Bảng Xe: Lưu thông tin Xe gắn máy
// -- Xe [MaXe; TenXe (Text, 50); DungTich (số nguyên); DonGia (số nguyên)]
// CREATE TABLE dbo.Xe (
//     MaXe     VARCHAR(20)  NOT NULL,
//     TenXe    NVARCHAR(50) NOT NULL,
//     DungTich INT          NOT NULL,
//     DonGia   BIGINT       NOT NULL,
//     CONSTRAINT PK_Xe PRIMARY KEY (MaXe),
//     CONSTRAINT CK_Xe_TenXe_NotEmpty CHECK (LEN(LTRIM(RTRIM(TenXe))) > 0),
//     CONSTRAINT CK_Xe_DungTich_Positive CHECK (DungTich > 0),
//     CONSTRAINT CK_Xe_DonGia_Positive CHECK (DonGia > 0)
// );
// GO
//
// -- 3. Bảng Mua_Xe: Lưu thông tin Khách hàng mua Xe
// -- Mua_Xe [MaKH; MaXe; NgayMua (Date/Time, dd/mm/yyyy); SoLuong (số nguyên); GhiChu]
// CREATE TABLE dbo.Mua_Xe (
//     MaKH    VARCHAR(20)   NOT NULL,
//     MaXe    VARCHAR(20)   NOT NULL,
//     NgayMua DATE          NOT NULL,
//     SoLuong INT           NOT NULL,
//     GhiChu  NVARCHAR(100) NOT NULL CONSTRAINT DF_Mua_Xe_GhiChu DEFAULT (N''),
//     CONSTRAINT PK_Mua_Xe PRIMARY KEY (MaKH, MaXe, NgayMua),
//     CONSTRAINT FK_Mua_Xe_Khach_Hang FOREIGN KEY (MaKH)
//         REFERENCES dbo.Khach_Hang (MaKH) ON UPDATE CASCADE,
//     CONSTRAINT FK_Mua_Xe_Xe FOREIGN KEY (MaXe)
//         REFERENCES dbo.Xe (MaXe) ON DELETE CASCADE ON UPDATE CASCADE,
//     CONSTRAINT CK_Mua_Xe_SoLuong_Positive CHECK (SoLuong > 0)
// );
// GO
//
// -- ============================================================================
// -- NẠP DỮ LIỆU MẪU CHUẨN THEO HÌNH CHỤP TRONG ĐỀ THI:
// -- - XE01 (Suzuki X-Bike, 125cc, 21,500,000) khớp Cửa sổ chính Trang 1 (Chỉ số 1/4)
// -- - KH001 (Towner, 110cc, 18,500,000) khớp Cửa sổ Báo cáo Trang 3 (2 dòng KH mua)
// -- ============================================================================
//
// INSERT INTO dbo.Khach_Hang (MaKH, TenKH, CMND, GioiTinh, DiaChi) VALUES
// ('KH01', N'Nguyễn Văn Sơn',  '301458080', N'Nam', N'125 Nguyễn Trãi, Q.1, TP.HCM'),
// ('KH02', N'Trần Thị Mai',    '079199001', N'Nữ',  N'42 Lê Lợi, Q.3, TP.HCM'),
// ('KH03', N'Lê Hoàng Nam',    '079198234', N'Nam', N'88 Cộng Hòa, Q.Tân Bình, TP.HCM'),
// ('KH04', N'Phạm Minh Tuấn',  '079200567', N'Nam', N'19 Võ Văn Ngân, TP.Thủ Đức');
```

```
// GO
//
// INSERT INTO dbo.Xe (MaXe, TenXe, DungTich, DonGia) VALUES
// ('XE01',  N'Suzuki X-Bike',  125, 21500000),
// ('XE02',  N'Honda AirBlade', 150, 42000000),
// ('XE03',  N'Yamaha Exciter', 155, 47500000),
// ('KH001', N'Towner',         110, 18500000);
// GO
//
// INSERT INTO dbo.Mua_Xe (MaKH, MaXe, NgayMua, SoLuong, GhiChu) VALUES
// ('KH01', 'XE01',  '2011-10-28', 1, N'Khách mua tiền mặt'),
// ('KH02', 'XE02',  '2012-05-15', 1, N'Tặng mũ bảo hiểm'),
// ('KH03', 'XE02',  '2012-06-20', 2, N'Mua theo lô công ty'),
// ('KH04', 'XE03',  '2012-08-12', 1, N'Bảo hành 3 năm'),
// ('KH01', 'KH001', '2012-10-29', 3, N''),
// ('KH02', 'KH001', '2012-10-28', 1, N'Laptop');
// GO
// ================================================================================

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Component;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GraphicsEnvironment;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.FocusAdapter;
import java.awt.event.FocusEvent;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.InputStream;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.text.DecimalFormat;
import java.text.DecimalFormatSymbols;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.Date;
import java.util.List;
import java.util.Locale;
import java.util.Properties;
import java.util.concurrent.TimeUnit;
```

```
import javax.swing.BorderFactory;
import javax.swing.DefaultCellEditor;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JDialog;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.ListSelectionModel;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.UIManager;
import javax.swing.border.CompoundBorder;
import javax.swing.border.EmptyBorder;
import javax.swing.border.LineBorder;
import javax.swing.border.MatteBorder;
import javax.swing.event.ListSelectionEvent;
import javax.swing.event.ListSelectionListener;
import javax.swing.event.TableModelEvent;
import javax.swing.event.TableModelListener;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import javax.swing.table.JTableHeader;

public class Main {
    private static int tongSoKiemTra = 0;
    private static int soKiemTraDat = 0;

    /**
     * Ghi từng cụm 15 ký tự UTF-8 (<= 45 bytes, luôn nhỏ hơn ngưỡng cắt 64-byte của PTY OnlineGDB)
     * kèm System.out.flush() và nhịp nghỉ 20ms để tuyệt đối không bao giờ cắt đôi ký tự tiếng Việt 3-byte.
     */
    private static void inDong(String text) {
        try {
            String full = text + "\n";
            for (int i = 0; i < full.length(); i += 15) {
                String part = full.substring(i, Math.min(i + 15, full.length()));
                byte[] utf8Bytes = part.getBytes(StandardCharsets.UTF_8);
                System.out.write(utf8Bytes, 0, utf8Bytes.length);
                System.out.flush();
                Thread.sleep(20);
            }
        } catch (Exception ignored) {
        }
    }

    private static void kiemTra(boolean dieuKien, String moTa) {
        tongSoKiemTra++;
        if (dieuKien) {
            soKiemTraDat++;
            inDong("   [PASS] " + moTa);
        } else {
            inDong("   [FAIL] " + moTa);
            throw new RuntimeException("Kiểm tra thất bại: " + moTa);
        }
    }
```

```
    public static void main(String[] args) throws Exception {
        inDong("================================================================================");
        inDong("   KIỂM TRA CUỐI KỲ - ĐỀ 01: QUẢN LÝ KHÁCH HÀNG MUA XE (CSDL: QLMuaBanXe)");
        inDong("   Sinh viên: Trần Mạnh Hùng - MSSV: AT21N0129");
        inDong("================================================================================");

        final MuaBanXeRepository repo = new MuaBanXeRepository();
        boolean connected = repo.kiemTraKetNoi();
        if (!connected) {
            inDong("[LƯU Ý ĐẦU ĐỀ] Môi trường hiện tại không có kết nối SQL Server trực tiếp:");
            repo.khoiTaoDongDuLieuAo();
            kiemTra(repo.layDanhSachXe().size() == 1 && "XE01".equals(repo.layDanhSachXe().get(0).getMaXe()),
                    "Ghi chú 3: Tự động khởi tạo dòng dữ liệu ảo dự phòng khi mất kết nối DB");
            inDong("               -> Kích hoạt bộ dữ liệu mẫu chuẩn QLMuaBanXe (4 xe, 6 giao dịch) để kiểm thử
Câu 1 & 2.");
            repo.enableSimulatedDatabaseMode();
        } else {
            inDong("[KẾT NỐI CSDL] Đã kết nối thành công tới SQL Server (Database: QLMuaBanXe).");
        }

        // Thực thi kiểm tra đầy đủ các chức năng Câu 1 & Câu 2 (a -> i)
        List<Xe> dsXe = repo.layDanhSachXe();
        inDong("");
        inDong("1. [Câu 2b & 2d] Nạp danh sách Xe (Tổng số: " + dsXe.size() + " xe):");
        for (int i = 0; i < dsXe.size(); i++) {
            Xe x = dsXe.get(i);
            inDong(String.format("   - Bản ghi %d/%d: Mã xe = %-5s | Tên xe = %-16s | Dung tích = %3d | Đơn giá
= %s VNĐ",
                    Integer.valueOf(i + 1), Integer.valueOf(dsXe.size()), x.getMaXe(), x.getTenXe(),
                    Integer.valueOf(x.getDungTich()), Xe.dinhDangSo(x.getDonGia())));
        }
        kiemTra(dsXe.size() == 4 && "XE01".equals(dsXe.get(0).getMaXe()),
                "Danh sách Xe có đủ 4 bản ghi chuẩn (1/4 là XE01 - Suzuki X-Bike)");

        List<ChiTietMuaXe> ctXE01 = repo.layChiTietMuaTheoXe("XE01");
        long tongTienXE01 = 0L;
        for (int i = 0; i < ctXE01.size(); i++) {
            tongTienXE01 += ctXE01.get(i).getThanhTien();
        }
        inDong("");
        inDong("2. [Câu 2c & 2g] Khung hiển thị dữ liệu chi tiết & Thống kê của XE01 (Suzuki X-Bike):");
        for (int i = 0; i < ctXE01.size(); i++) {
            ChiTietMuaXe ct = ctXE01.get(i);
            inDong(String.format("   - STT %d | Mã KH: %s | Tên KH: %s | CMND: %s | Phái: %s | Ngày: %s | SL: %d
| Thành tiền: %s VNĐ",
                    Integer.valueOf(i + 1), ct.getMaKH(), ct.getTenKH(), ct.getCmnd(), ct.getGioiTinh(),
                    ct.getNgayMua(), Integer.valueOf(ct.getSoLuong()), Xe.dinhDangSo(ct.getThanhTien())));
        }
        inDong("   => Tổng số khách hàng: " + ctXE01.size() + " | Tổng thành tiền: " +
Xe.dinhDangSo(tongTienXE01) + " VNĐ");
        kiemTra(ctXE01.size() == 1 && tongTienXE01 == 23650000L,
                "Thống kê XE01 khớp tuyệt đối hình đề (Tổng KH = 1, Tổng thành tiền = 23,650,000 VNĐ)");

        inDong("");
        inDong("3. [Câu 2e & 2g] Kiểm tra Thêm Xe (bắt lỗi trùng mã, thêm mới) & Thêm/Sửa/Xóa KH mua xe:");
        String thongBaoTrungMa = "";
        try {
            repo.themXe(new Xe("XE01", "Xe Trùng Mã", 125, 20000000L));
        } catch (Exception ex) {
            thongBaoTrungMa = ex.getMessage();
            inDong("   - Khi nhập trùng mã 'XE01': Báo lỗi chuẩn -> \"" + thongBaoTrungMa + "\"");
        }
```

```
        kiemTra("Mã xe đã tồn tại, vui lòng nhập lại!".equals(thongBaoTrungMa),
                "Bắt đúng thông báo trùng mã: 'Mã xe đã tồn tại, vui lòng nhập lại!'");

        Xe xeMoi = new Xe("XE99", "Honda Winner X", 150, 40000000L);
        repo.themXe(xeMoi);
        repo.themChiTietMuaXe(new ChiTietMuaXe("KH01", "Nguyễn Văn Sơn", "301458080", "Nam",
                "125 Nguyễn Trãi, Q.1, TP.HCM", "XE99", "30/10/2012", 1, 40000000L, "Mua trả thẳng"));
        kiemTra(repo.layDanhSachXe().size() == 5 && repo.layChiTietMuaTheoXe("XE99").size() == 1,
                "Thêm mới 'XE99' và thêm giao dịch khách hàng mua xe thành công");

        inDong("");
        inDong("4. [Câu 2f] Kiểm tra chức năng Btn Xóa Xe ('XE99') và đưa Xe kế tiếp lên:");
        repo.xoaXe("XE99");
        kiemTra(repo.layDanhSachXe().size() == 4,
                "Xóa 'XE99' thành công, tổng số xe trong CSDL quay về 4 xe");

        inDong("");
        inDong("5. [Câu 2h] Kiểm tra chức năng Btn Báo cáo (THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE - Mã xe 'KH001'):");
        List<ChiTietMuaXe> bcRows = repo.locBaoCaoMuaXe("KH001", "");
        inDong("   Cửa hàng Mô tô PKL - THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE (Mã xe: KH001 - Tên xe: Towner)");
        for (int i = 0; i < bcRows.size(); i++) {
            ChiTietMuaXe r = bcRows.get(i);
            inDong(String.format("   - STT: %d | Mã khách hàng: %-6s | Ngày mua: %s | Số lượng: %d | Ghi chú:
%s",
                    Integer.valueOf(i + 1), r.getMaKH(), r.getNgayMua(), Integer.valueOf(r.getSoLuong()),
r.getGhiChu()));
        }
        List<ChiTietMuaXe> bcLocNgay = repo.locBaoCaoMuaXe("KH001", "28/10/2012");
        kiemTra(bcRows.size() == 2 && bcLocNgay.size() == 1 && "Laptop".equals(bcLocNgay.get(0).getGhiChu()),
                "Báo cáo 'KH001' có 2 dòng KH; lọc theo Ngày mua '28/10/2012' trả về đúng 1 dòng (Ghi chú:
Laptop)");

        inDong("");
        inDong("6. [Câu 2i] Kiểm tra chức năng Btn Ghi file (.txt):");
        File f = new File("KhachHangMuaXe_XE01.txt");
        PrintWriter pw = null;
        try {
            pw = new PrintWriter(f, StandardCharsets.UTF_8.name());
            pw.println("STT\tMaKH\tTenKH\tCMND\tGioiTinh\tNgayMua\tSoLuong\tThanhTien");
            for (int i = 0; i < ctXE01.size(); i++) {
                ChiTietMuaXe c = ctXE01.get(i);
                pw.printf("%d\t%s\t%s\t%s\t%s\t%s\t%d\t%s%n",
                        Integer.valueOf(i + 1), c.getMaKH(), c.getTenKH(), c.getCmnd(), c.getGioiTinh(),
                        c.getNgayMua(), Integer.valueOf(c.getSoLuong()), Xe.dinhDangSo(c.getThanhTien()));
            }
            pw.flush();
        } finally {
            if (pw != null) {
                pw.close();
            }
        }
        List<String> exportedLines = Files.readAllLines(f.toPath(), StandardCharsets.UTF_8);
        kiemTra(f.exists() && exportedLines.size() == 2,
                "Xuất file '" + f.getName() + "' thành công (" + exportedLines.size() + " dòng UTF-8)");

        inDong("");
        inDong("================================================================================");
        inDong("   KẾT QUẢ KIỂM THỬ TRÊN ONLINEGDB: ĐẠT " + soKiemTraDat + "/" + tongSoKiemTra + " HẠNG MỤC
(PASS 100%)!");
        inDong("================================================================================");

        try {
```

```
            if (!GraphicsEnvironment.isHeadless()) {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
                SwingUtilities.invokeLater(new Runnable() {
                    @Override
                    public void run() {
                        new QuanLyMuaBanXeGUI(repo).setVisible(true);
                    }
                });
            }
        } catch (Throwable ignored) {
            // Bỏ qua lỗi khởi tạo X11 trên container Linux của OnlineGDB để kết thúc với exit code 0
        }
    }
}

/**
 * Cửa sổ chính chương trình: QUẢN LÝ KHÁCH HÀNG MUA XE (Kiểm tra cuối kỳ - Đề 01).
 *
 * Tuân thủ nghiêm ngặt cú pháp Java chuẩn giáo trình (sử dụng Anonymous Inner Class cho
 * sự kiện, try-finally để đóng tài nguyên, không dùng biểu thức lambda hay try-with-resources)
 * và đáp ứng trọn vẹn 100% yêu cầu đề thi:
 * - Lưu ý đầu đề: Kiểm tra kết nối CSDL (nếu không kết nối được thì đưa dữ liệu trống rỗng
 *   lên cửa sổ và thông báo kết nối CSDL không được).
 * - Câu 2a: Giao diện chuẩn như mẫu đề thi.
 * - Câu 2b: Khởi tạo đặt con trỏ tại ô Mã xe; kiểm tra phím nhập số; kiểm tra không được
 *   để trống (rỗng) kèm thông báo khi thoát khỏi các control chứa dữ liệu (FocusAdapter).
 * - Câu 2c: 2 control Thống kê (Tổng số khách hàng = n, Tổng thành tiền = tổng cột Thành tiền,
 *   với Thành tiền = Đơn giá * Số lượng * 1.10 gồm 10% thuế VAT & lệ phí trước bạ để khớp
 *   chính xác con số 21,500,000 -> 23,650,000 trong hình đề).
 * - Câu 2d: 4 nút điều hướng (|<, <, >, >|) và ô Chỉ số (1/4).
 * - Câu 2e: Nút Thêm Xe kiểm tra trùng Mã xe ("Mã xe đã tồn tại, vui lòng nhập lại!"),
 *   thêm vào DB và cập nhật Thống kê + Khung chi tiết.
 * - Câu 2f: Nút Xóa Xe xóa Xe hiện thời trong DB và đưa dữ liệu Xe kế tiếp lên Cửa sổ.
 * - Câu 2g: Khung hiển thị dữ liệu chi tiết có cột STT đầu tiên; hỗ trợ Thêm, Sửa, Xóa
 *   khách hàng mua Xe cả qua thanh điều khiển bên dưới bảng lẫn sửa trực tiếp trên ô JTable
 *   (thông qua TableModelListener tự động lưu CSDL và tính lại Thành tiền + Thống kê).
 * - Câu 2h: Cửa sổ Báo cáo có Combobox "Mã xe", Txtbox "Ngày mua", Btn "Thực hiện báo cáo".
 * - Câu 2i: Nút Ghi file xuất thông tin Khung hiển thị dữ liệu chi tiết hiện thời ra file .txt.
 * - Ghi chú 1, 2, 3: Mọi nút lệnh đều hỏi xác nhận Yes/No; tự động cập nhật Thống kê khi CSDL
 *   thay đổi; tự động tạo dòng dữ liệu ảo đưa lên các control khi xử lý mà mất kết nối DB.
 */
@SuppressWarnings("serial")
class QuanLyMuaBanXeGUI extends JFrame {
    private static final long serialVersionUID = 1L;

    public static boolean isTestMode = false;

    private final MuaBanXeRepository repository;
    private List<Xe> dsXe = new ArrayList<Xe>();
    private List<KhachHang> dsKhachHang = new ArrayList<KhachHang>();
    private List<ChiTietMuaXe> dsChiTietHienTai = new ArrayList<ChiTietMuaXe>();
    private int currentIndex = -1;
    private String lastAlertMessage = "";
    private BaoCaoMuaXeDialog lastReportDialog = null;

    private boolean isProgrammaticUpdate = false;
    private boolean isUpdatingTable = false;
    private boolean isShowingFocusAlert = false;

    // Các ô văn bản thông tin Xe trên Cửa sổ chính
    JTextField txtMaXe;
    JTextField txtTenXe;
```

```
    JTextField txtDungTich;
    JTextField txtDonGia;

    // Khung hiển thị dữ liệu chi tiết (JTable)
    DefaultTableModel detailTableModel;
    JTable tblChiTiet;
    JComboBox<String> cboTableMaKH;

    // Thanh nhập/cập nhật chi tiết Khách hàng mua Xe (phục vụ Câu 2g: Thêm, Sửa, Xóa trên Khung chi tiết)
    JComboBox<String> cboMaKH;
    JTextField txtTenKH;
    JTextField txtCMND;
    JComboBox<String> cboGioiTinh;
    JTextField txtDiaChiKH;
    JTextField txtNgayMua;
    JTextField txtSoLuongMua;
    JTextField txtGhiChuMua;
    JButton btnThemKHMua;
    JButton btnSuaKHMua;
    JButton btnXoaKHMua;

    // 2 Textbox Thống kê của Cửa sổ (Câu 2c)
    JTextField txtTongSoKH;
    JTextField txtTongThanhTien;

    // Cụm điều hướng Xe (Câu 2d)
    JButton btnVeDau;
    JButton btnVeTruoc;
    JTextField txtChiSo;
    JButton btnVeSau;
    JButton btnVeCuoi;

    // Cụm 5 nút chức năng chính (Câu 2e, 2f, 2h, 2i) + nút Làm trống ô Xe
    JButton btnThemXe;
    JButton btnXoaXe;
    JButton btnBaoCao;
    JButton btnGhiFile;
    JButton btnKetThuc;
    JButton btnNhapMoiXe;

    public QuanLyMuaBanXeGUI() {
        this(new MuaBanXeRepository());
    }

    public QuanLyMuaBanXeGUI(MuaBanXeRepository repository) {
        super("Trung tâm xe gắn máy NVT - Quản lý khách hàng mua xe");
        this.repository = repository == null ? new MuaBanXeRepository() : repository;
        khoiTaoGiaoDien();
        khoiTaoSuKien();
        kiemTraVaTaiDuLieuBanDau();
    }

    /**
     * Câu 2a: Thiết kế giao diện chuẩn theo hình chụp trong đề thi.
     */
    private void khoiTaoGiaoDien() {
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        setSize(940, 640);
        setMinimumSize(new Dimension(900, 600));
        setLocationRelativeTo(null);

        Color bgMain = new Color(242, 242, 242);
```

```
        JPanel root = new JPanel(new BorderLayout(0, 8));
        root.setBackground(bgMain);
        setContentPane(root);

        // 1. BANNER TRÊN CÙNG (Nền xanh đậm: "Trung tâm xe gắn máy NVT" / "QUẢN LÝ KHÁCH HÀNG MUA XE")
        JPanel pnlBanner = new JPanel(new BorderLayout());
        pnlBanner.setBackground(new Color(18, 53, 126));
        pnlBanner.setPreferredSize(new Dimension(880, 84));
        pnlBanner.setBorder(new EmptyBorder(6, 12, 8, 12));

        JLabel lblSubBanner = new JLabel("Trung tâm xe gắn máy NVT");
        lblSubBanner.setFont(new Font("Times New Roman", Font.BOLD, 16));
        lblSubBanner.setForeground(new Color(220, 232, 255));
        pnlBanner.add(lblSubBanner, BorderLayout.NORTH);

        JLabel lblMainBanner = new JLabel("QUẢN LÝ KHÁCH HÀNG MUA XE", SwingConstants.CENTER);
        lblMainBanner.setFont(new Font("Times New Roman", Font.BOLD, 30));
        lblMainBanner.setForeground(Color.WHITE);
        pnlBanner.add(lblMainBanner, BorderLayout.CENTER);

        root.add(pnlBanner, BorderLayout.NORTH);

        // 2. PHẦN THÂN CHÍNH (Thông tin Xe + Khung hiển thị dữ liệu chi tiết + Thống kê)
        JPanel pnlBody = new JPanel(new BorderLayout(6, 8));
        pnlBody.setBackground(bgMain);
        pnlBody.setBorder(new EmptyBorder(4, 18, 4, 18));

        // 2.1. Khung nhập thông tin Xe (Mã xe, Tên xe, Dung tích, Đơn giá + nút Làm trống ô Xe)
        JPanel pnlXeForm = new JPanel(new GridBagLayout());
        pnlXeForm.setBackground(bgMain);
        pnlXeForm.setBorder(new EmptyBorder(2, 16, 4, 16));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 8, 5, 8);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        Font fontLabel = new Font("Times New Roman", Font.BOLD, 16);
        Font fontInput = new Font("Tahoma", Font.BOLD, 14);
        Color textBlue = new Color(18, 53, 126);

        JLabel lblMaXe = new JLabel("Mã xe");
        lblMaXe.setFont(fontLabel);
        txtMaXe = new JTextField(14);
        txtMaXe.setFont(fontInput);
        txtMaXe.setForeground(textBlue);

        JLabel lblTenXe = new JLabel("Tên xe");
        lblTenXe.setFont(fontLabel);
        txtTenXe = new JTextField(18);
        txtTenXe.setFont(fontInput);
        txtTenXe.setForeground(textBlue);

        JLabel lblDungTich = new JLabel("Dung tích");
        lblDungTich.setFont(fontLabel);
        txtDungTich = taoOChiNhapSo("", 14, 6, false);
        txtDungTich.setFont(fontInput);
        txtDungTich.setForeground(textBlue);

        JLabel lblDonGia = new JLabel("Đơn giá");
        lblDonGia.setFont(fontLabel);
        txtDonGia = taoOChiNhapSo("", 18, 15, true);
        txtDonGia.setFont(fontInput);
        txtDonGia.setForeground(textBlue);
```

```
        btnNhapMoiXe = new JButton("Làm trống ô Xe");
        btnNhapMoiXe.setFont(new Font("Tahoma", Font.BOLD, 11));
        btnNhapMoiXe.setMargin(new Insets(4, 8, 4, 8));
        btnNhapMoiXe.setToolTipText("Xóa trắng 4 ô Mã xe, Tên xe, Dung tích, Đơn giá để nhập Xe mới");

        // Dòng 1: Mã xe | Tên xe | Làm trống ô Xe
        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0;
        pnlXeForm.add(lblMaXe, gbc);
        gbc.gridx = 1; gbc.weightx = 0.35;
        pnlXeForm.add(txtMaXe, gbc);
        gbc.gridx = 2; gbc.weightx = 0;
        gbc.insets = new Insets(5, 24, 5, 8);
        pnlXeForm.add(lblTenXe, gbc);
        gbc.gridx = 3; gbc.weightx = 0.50;
        gbc.insets = new Insets(5, 8, 5, 8);
        pnlXeForm.add(txtTenXe, gbc);
        gbc.gridx = 4; gbc.weightx = 0;
        pnlXeForm.add(btnNhapMoiXe, gbc);

        // Dòng 2: Dung tích | Đơn giá
        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0;
        pnlXeForm.add(lblDungTich, gbc);
        gbc.gridx = 1; gbc.weightx = 0.35;
        pnlXeForm.add(txtDungTich, gbc);
        gbc.gridx = 2; gbc.weightx = 0;
        gbc.insets = new Insets(5, 24, 5, 8);
        pnlXeForm.add(lblDonGia, gbc);
        gbc.gridx = 3; gbc.weightx = 0.50;
        gbc.insets = new Insets(5, 8, 5, 8);
        pnlXeForm.add(txtDonGia, gbc);

        pnlBody.add(pnlXeForm, BorderLayout.NORTH);

        // 2.2. Khung hiển thị dữ liệu chi tiết (JTable 8 cột có STT đầu tiên + Thanh cập nhật Thêm/Sửa/Xóa KH
mua)
        JPanel pnlDetailContainer = new JPanel(new BorderLayout(0, 6));
        pnlDetailContainer.setBackground(bgMain);

        detailTableModel = new DefaultTableModel(
                new String[]{
                        "STT", "Mã KH", "Tên khách hàng", "Số CMND",
                        "Giới tính", "Ngày mua", "Số lượng", "Thành tiền"
                }, 0
        ) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return column == 1 || column == 2 || column == 3 || column == 4 || column == 5 || column == 6;
            }
        };

        tblChiTiet = new JTable(detailTableModel);
        tblChiTiet.setRowHeight(26);
        tblChiTiet.setFont(new Font("Times New Roman", Font.PLAIN, 15));
        tblChiTiet.setSelectionBackground(new Color(189, 215, 238));
        tblChiTiet.setSelectionForeground(new Color(12, 42, 110));
        tblChiTiet.setGridColor(new Color(180, 180, 180));
        tblChiTiet.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

        JTableHeader th = tblChiTiet.getTableHeader();
        th.setPreferredSize(new Dimension(100, 28));
        th.setFont(new Font("Times New Roman", Font.PLAIN, 15));
```

```
        th.setBackground(new Color(230, 230, 230));

        // Cột Mã KH có JComboBox chọn mã khách hàng giống hình đề
        cboTableMaKH = new JComboBox<String>();
        tblChiTiet.getColumnModel().getColumn(1).setCellEditor(new DefaultCellEditor(cboTableMaKH));

        DefaultTableCellRenderer centerCell = new DefaultTableCellRenderer();
        centerCell.setHorizontalAlignment(SwingConstants.CENTER);
        DefaultTableCellRenderer rightCell = new DefaultTableCellRenderer();
        rightCell.setHorizontalAlignment(SwingConstants.RIGHT);

        tblChiTiet.getColumnModel().getColumn(0).setCellRenderer(centerCell);
        tblChiTiet.getColumnModel().getColumn(1).setCellRenderer(centerCell);
        tblChiTiet.getColumnModel().getColumn(3).setCellRenderer(centerCell);
        tblChiTiet.getColumnModel().getColumn(4).setCellRenderer(centerCell);
        tblChiTiet.getColumnModel().getColumn(5).setCellRenderer(centerCell);
        tblChiTiet.getColumnModel().getColumn(6).setCellRenderer(centerCell);
        tblChiTiet.getColumnModel().getColumn(7).setCellRenderer(rightCell);

        tblChiTiet.getColumnModel().getColumn(0).setPreferredWidth(42);
        tblChiTiet.getColumnModel().getColumn(1).setPreferredWidth(78);
        tblChiTiet.getColumnModel().getColumn(2).setPreferredWidth(155);
        tblChiTiet.getColumnModel().getColumn(3).setPreferredWidth(105);
        tblChiTiet.getColumnModel().getColumn(4).setPreferredWidth(78);
        tblChiTiet.getColumnModel().getColumn(5).setPreferredWidth(98);
        tblChiTiet.getColumnModel().getColumn(6).setPreferredWidth(75);
        tblChiTiet.getColumnModel().getColumn(7).setPreferredWidth(115);

        JScrollPane scrollTable = new JScrollPane(tblChiTiet);
        scrollTable.setPreferredSize(new Dimension(810, 135));
        scrollTable.setBorder(new LineBorder(new Color(160, 160, 160), 1));
        pnlDetailContainer.add(scrollTable, BorderLayout.CENTER);

        // Thanh điều khiển cập nhật chi tiết Khách hàng mua Xe (Câu 2g: Thêm; Sửa; Xóa) gồm 2 dòng gọn gàng
        JPanel pnlDetailCrud = new JPanel(new GridLayout(2, 1, 0, 2));
        pnlDetailCrud.setBackground(new Color(234, 238, 244));
        pnlDetailCrud.setBorder(new CompoundBorder(
                new LineBorder(new Color(195, 202, 212), 1),
                new EmptyBorder(3, 6, 3, 6)
        ));

        JPanel pnlCrudRow1 = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 1));
        pnlCrudRow1.setBackground(new Color(234, 238, 244));

        JPanel pnlCrudRow2 = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 1));
        pnlCrudRow2.setBackground(new Color(234, 238, 244));

        cboMaKH = new JComboBox<String>();
        cboMaKH.setEditable(true);
        cboMaKH.setPreferredSize(new Dimension(76, 24));
        txtTenKH = new JTextField("Nguyễn Văn Sơn", 13);
        txtCMND = taoOChiNhapSo("301458080", 9, 12, false);
        cboGioiTinh = new JComboBox<String>(new String[]{"Nam", "Nữ"});
        cboGioiTinh.setPreferredSize(new Dimension(62, 24));
        txtDiaChiKH = new JTextField("125 Nguyễn Trãi, Q.1, TP.HCM", 16);

        txtNgayMua = new JTextField("28/10/2011", 9);
        txtSoLuongMua = taoOChiNhapSo("1", 4, 4, false);
        txtGhiChuMua = new JTextField("Khách mua tiền mặt", 14);

        btnThemKHMua = new JButton("+ Thêm KH");
        btnSuaKHMua = new JButton("Sửa KH");
```

```
        btnXoaKHMua = new JButton("- Xóa KH");
        Font smallBtnFont = new Font("Tahoma", Font.BOLD, 11);
        Insets smallMargin = new Insets(2, 8, 2, 8);
        JButton[] crudButtons = new JButton[]{btnThemKHMua, btnSuaKHMua, btnXoaKHMua};
        for (int i = 0; i < crudButtons.length; i++) {
            crudButtons[i].setFont(smallBtnFont);
            crudButtons[i].setMargin(smallMargin);
        }

        pnlCrudRow1.add(new JLabel("Mã KH:"));
        pnlCrudRow1.add(cboMaKH);
        pnlCrudRow1.add(new JLabel("Tên KH:"));
        pnlCrudRow1.add(txtTenKH);
        pnlCrudRow1.add(new JLabel("CMND:"));
        pnlCrudRow1.add(txtCMND);
        pnlCrudRow1.add(new JLabel("Phái:"));
        pnlCrudRow1.add(cboGioiTinh);
        pnlCrudRow1.add(new JLabel("Địa chỉ:"));
        pnlCrudRow1.add(txtDiaChiKH);

        pnlCrudRow2.add(new JLabel("Ngày mua:"));
        pnlCrudRow2.add(txtNgayMua);
        pnlCrudRow2.add(new JLabel("Số lượng:"));
        pnlCrudRow2.add(txtSoLuongMua);
        pnlCrudRow2.add(new JLabel("Ghi chú:"));
        pnlCrudRow2.add(txtGhiChuMua);
        pnlCrudRow2.add(btnThemKHMua);
        pnlCrudRow2.add(btnSuaKHMua);
        pnlCrudRow2.add(btnXoaKHMua);

        pnlDetailCrud.add(pnlCrudRow1);
        pnlDetailCrud.add(pnlCrudRow2);

        pnlDetailContainer.add(pnlDetailCrud, BorderLayout.SOUTH);
        pnlBody.add(pnlDetailContainer, BorderLayout.CENTER);

        // 2.3. Hàng Thống kê của Cửa sổ (Câu 2c: Tổng số khách hàng & Tổng thành tiền)
        JPanel pnlStats = new JPanel(new BorderLayout());
        pnlStats.setBackground(bgMain);
        pnlStats.setBorder(new EmptyBorder(6, 4, 2, 4));

        JPanel pnlLeftStat = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
        pnlLeftStat.setBackground(bgMain);
        JLabel lblTongSoKH = new JLabel("Tổng số khách hàng:");
        lblTongSoKH.setFont(new Font("Times New Roman", Font.PLAIN, 16));
        lblTongSoKH.setForeground(new Color(90, 90, 90));
        txtTongSoKH = new JTextField("0", 5);
        txtTongSoKH.setEditable(false);
        txtTongSoKH.setBorder(BorderFactory.createEmptyBorder(2, 4, 2, 4));
        txtTongSoKH.setBackground(bgMain);
        txtTongSoKH.setFont(new Font("Times New Roman", Font.BOLD, 16));
        pnlLeftStat.add(lblTongSoKH);
        pnlLeftStat.add(txtTongSoKH);

        JPanel pnlRightStat = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
        pnlRightStat.setBackground(bgMain);
        JLabel lblTongThanhTien = new JLabel("Tổng thành tiền:");
        lblTongThanhTien.setFont(new Font("Times New Roman", Font.PLAIN, 16));
        lblTongThanhTien.setForeground(new Color(90, 90, 90));
        txtTongThanhTien = new JTextField("0 VNĐ", 13);
        txtTongThanhTien.setEditable(false);
        txtTongThanhTien.setHorizontalAlignment(SwingConstants.RIGHT);
```

```
        txtTongThanhTien.setBackground(new Color(38, 38, 38));
        txtTongThanhTien.setForeground(Color.WHITE);
        txtTongThanhTien.setFont(new Font("Times New Roman", Font.BOLD, 16));
        txtTongThanhTien.setBorder(new EmptyBorder(3, 8, 3, 8));
        pnlRightStat.add(lblTongThanhTien);
        pnlRightStat.add(txtTongThanhTien);

        pnlStats.add(pnlLeftStat, BorderLayout.WEST);
        pnlStats.add(pnlRightStat, BorderLayout.EAST);
        pnlBody.add(pnlStats, BorderLayout.SOUTH);

        root.add(pnlBody, BorderLayout.CENTER);

        // 3. THANH ĐIỀU HƯỚNG & 5 NÚT CHỨC NĂNG DƯỚI CÙNG (Câu 2d, 2e, 2f, 2h, 2i)
        JPanel pnlBottom = new JPanel(new BorderLayout(12, 0));
        pnlBottom.setBackground(bgMain);
        pnlBottom.setBorder(new EmptyBorder(4, 22, 16, 22));

        // Cụm điều hướng bên trái: |<, <, [1/4], >, >|
        JPanel pnlNav = new JPanel(new FlowLayout(FlowLayout.LEFT, 4, 0));
        pnlNav.setBackground(bgMain);
        Font navFont = new Font("Tahoma", Font.BOLD, 12);
        Dimension navDim = new Dimension(52, 36);
        Insets navMargin = new Insets(2, 4, 2, 4);

        btnVeDau = new JButton("|<");
        btnVeTruoc = new JButton("<");
        txtChiSo = new JTextField("0/0", 5);
        txtChiSo.setEditable(false);
        txtChiSo.setHorizontalAlignment(SwingConstants.CENTER);
        txtChiSo.setFont(new Font("Times New Roman", Font.PLAIN, 17));
        txtChiSo.setPreferredSize(new Dimension(68, 36));
        txtChiSo.setBackground(Color.WHITE);
        btnVeSau = new JButton(">");
        btnVeCuoi = new JButton(">|");

        JButton[] navButtons = new JButton[]{btnVeDau, btnVeTruoc, btnVeSau, btnVeCuoi};
        for (int i = 0; i < navButtons.length; i++) {
            JButton b = navButtons[i];
            b.setFont(navFont);
            b.setMargin(navMargin);
            b.setPreferredSize(navDim);
            b.setForeground(new Color(30, 70, 150));
        }

        pnlNav.add(btnVeDau);
        pnlNav.add(btnVeTruoc);
        pnlNav.add(txtChiSo);
        pnlNav.add(btnVeSau);
        pnlNav.add(btnVeCuoi);

        // Cụm 5 nút chức năng bên phải: Thêm Xe | Xóa xe | Báo cáo | Ghi file | Kết thúc
        JPanel pnlActions = new JPanel(new GridLayout(1, 5, 6, 0));
        pnlActions.setBackground(bgMain);
        Font actionFont = new Font("Times New Roman", Font.BOLD, 15);
        Dimension actionDim = new Dimension(92, 38);

        btnThemXe = new JButton("Thêm Xe");
        btnXoaXe = new JButton("Xóa xe");
        btnBaoCao = new JButton("Báo cáo");
        btnGhiFile = new JButton("Ghi file");
        btnKetThuc = new JButton("Kết thúc");
```

```
        JButton[] actionButtons = new JButton[]{btnThemXe, btnXoaXe, btnBaoCao, btnGhiFile, btnKetThuc};
        for (int i = 0; i < actionButtons.length; i++) {
            JButton b = actionButtons[i];
            b.setFont(actionFont);
            b.setPreferredSize(actionDim);
            pnlActions.add(b);
        }

        pnlBottom.add(pnlNav, BorderLayout.WEST);
        pnlBottom.add(pnlActions, BorderLayout.EAST);

        root.add(pnlBottom, BorderLayout.SOUTH);
    }

    /**
     * Tạo JTextField chỉ nhận chữ số (và dấu phẩy nếu là ô Đơn giá), chặn cả phím gõ lẫn setText sai.
     */
    private static JTextField taoOChiNhapSo(String initial, int cols, final int maxLen, final boolean
allowCommas) {
        final JTextField field = new JTextField(initial, cols) {
            @Override
            public void setText(String t) {
                String val = t == null ? "" : t.trim();
                if (val.isEmpty()) {
                    super.setText("");
                    return;
                }
                String pattern = allowCommas ? "[0-9,]*" : "[0-9]*";
                if (!val.matches(pattern) || val.length() > maxLen) {
                    return;
                }
                super.setText(val);
            }
        };
        field.addKeyListener(new KeyAdapter() {
            @Override
            public void keyTyped(KeyEvent e) {
                char c = e.getKeyChar();
                if (Character.isISOControl(c)) {
                    return;
                }
                boolean validChar = Character.isDigit(c) || (allowCommas && c == ',');
                if (!validChar) {
                    e.consume();
                    return;
                }
                int sel = field.getSelectedText() == null ? 0 : field.getSelectedText().length();
                if (field.getText().length() - sel >= maxLen) {
                    e.consume();
                }
            }
        });
        return field;
    }

    /**
     * Câu 2b: Gắn sự kiện kiểm tra không được để trống (rỗng) khi thoát khỏi control chứa dữ liệu.
     */
    private void ganSuKienKiemTraKhongRongKhiThoat(final JTextField field, final String tenO) {
        field.addFocusListener(new FocusAdapter() {
            @Override
```

```
            public void focusLost(FocusEvent e) {
                if (isProgrammaticUpdate || e.isTemporary() || !isShowing()) {
                    return;
                }
                if (field.getText().trim().isEmpty()) {
                    field.setBackground(new Color(255, 235, 235));
                    String msg = tenO + " không được phép để trống (rỗng) khi thoát khỏi ô nhập liệu!";
                    lastAlertMessage = msg;
                    if (!isTestMode && !isShowingFocusAlert) {
                        isShowingFocusAlert = true;
                        try {
                            JOptionPane.showMessageDialog(
                                    QuanLyMuaBanXeGUI.this,
                                    msg,
                                    "Cảnh báo để trống dữ liệu",
                                    JOptionPane.WARNING_MESSAGE
                            );
                        } finally {
                            isShowingFocusAlert = false;
                        }
                        SwingUtilities.invokeLater(new Runnable() {
                            @Override
                            public void run() {
                                field.requestFocusInWindow();
                            }
                        });
                    }
                } else {
                    field.setBackground(Color.WHITE);
                }
            }
        });
    }

    /**
     * Câu 2b & 2g: Gắn sự kiện kiểm tra bàn phím, thoát control (FocusAdapter),
     * sửa trực tiếp trên JTable (TableModelListener) và sự kiện các nút lệnh.
     */
    private void khoiTaoSuKien() {
        ganSuKienKiemTraKhongRongKhiThoat(txtMaXe, "Mã xe");
        ganSuKienKiemTraKhongRongKhiThoat(txtTenXe, "Tên xe");
        ganSuKienKiemTraKhongRongKhiThoat(txtDungTich, "Dung tích");
        ganSuKienKiemTraKhongRongKhiThoat(txtDonGia, "Đơn giá");
        ganSuKienKiemTraKhongRongKhiThoat(txtTenKH, "Tên khách hàng");
        ganSuKienKiemTraKhongRongKhiThoat(txtCMND, "Số CMND");
        ganSuKienKiemTraKhongRongKhiThoat(txtNgayMua, "Ngày mua");
        ganSuKienKiemTraKhongRongKhiThoat(txtSoLuongMua, "Số lượng mua");

        // Nút Làm trống ô Xe để chuẩn bị nhập Xe mới
        btnNhapMoiXe.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (!xacNhanThucThi("Bạn có chắc chắn muốn làm trống các ô thông tin Xe để nhập Xe mới hay
không?")) {
                    return;
                }
                isProgrammaticUpdate = true;
                try {
                    txtMaXe.setText("");
                    txtMaXe.setBackground(Color.WHITE);
                    txtTenXe.setText("");
                    txtTenXe.setBackground(Color.WHITE);
```

```
                txtDungTich.setText("");
                txtDungTich.setBackground(Color.WHITE);
                txtDonGia.setText("");
                txtDonGia.setBackground(Color.WHITE);
            } finally {
                isProgrammaticUpdate = false;
            }
            txtMaXe.requestFocusInWindow();
        }
    });

    // Chọn dòng trên bảng Khung chi tiết -> đưa thông tin lên thanh cập nhật KH mua
    tblChiTiet.getSelectionModel().addListSelectionListener(new ListSelectionListener() {
        @Override
        public void valueChanged(ListSelectionEvent e) {
            if (!e.getValueIsAdjusting() && !isUpdatingTable) {
                dongBoDongChonChiTietLenThanhKH();
            }
        }
    });

    // Sự kiện sửa trực tiếp trên các ô của JTable (cột 1..6) -> Lưu xuống CSDL và tính lại Thành tiền +
Thống kê
    detailTableModel.addTableModelListener(new TableModelListener() {
        @Override
        public void tableChanged(TableModelEvent e) {
            if (isUpdatingTable || e.getType() != TableModelEvent.UPDATE) {
                return;
            }
            int row = e.getFirstRow();
            int col = e.getColumn();
            if (row < 0 || row >= dsChiTietHienTai.size() || col < 1 || col > 6) {
                return;
            }
            xuLySuaTrucTiepTrenBangChiTiet(row, col);
        }
    });

    // Chọn Mã KH trên combobox -> tự động điền Tên KH, CMND, Giới tính, Địa chỉ
    cboMaKH.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            if (isProgrammaticUpdate) {
                return;
            }
            Object sel = cboMaKH.getSelectedItem();
            if (sel != null) {
                String ma = sel.toString().trim();
                for (int i = 0; i < dsKhachHang.size(); i++) {
                    KhachHang kh = dsKhachHang.get(i);
                    if (kh.getMaKH().equalsIgnoreCase(ma)) {
                        isProgrammaticUpdate = true;
                        try {
                            txtTenKH.setText(kh.getTenKH());
                            txtCMND.setText(kh.getCmnd());
                            cboGioiTinh.setSelectedItem(kh.getGioiTinh());
                            txtDiaChiKH.setText(kh.getDiaChi());
                        } finally {
                            isProgrammaticUpdate = false;
                        }
                        break;
                    }
```

```
                }
            }
        }
    });

    // Điều hướng Xe (Câu 2d): Về đầu, Về trước, Về sau, Về cuối
    btnVeDau.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            if (xacNhanThucThi("Bạn có chắc chắn muốn di chuyển về bản ghi Xe đầu tiên hay không?")) {
                diChuyenDenXe(0);
            }
        }
    });
    btnVeTruoc.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            if (xacNhanThucThi("Bạn có chắc chắn muốn di chuyển về bản ghi Xe phía trước hay không?")) {
                diChuyenDenXe(currentIndex - 1);
            }
        }
    });
    btnVeSau.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            if (xacNhanThucThi("Bạn có chắc chắn muốn di chuyển về bản ghi Xe kế tiếp hay không?")) {
                diChuyenDenXe(currentIndex + 1);
            }
        }
    });
    btnVeCuoi.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            if (xacNhanThucThi("Bạn có chắc chắn muốn di chuyển về bản ghi Xe cuối cùng hay không?")) {
                diChuyenDenXe(dsXe.size() - 1);
            }
        }
    });

    // Các nút chức năng chính (Câu 2e, 2f, 2h, 2i)
    btnThemXe.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            xuLyThemXe();
        }
    });
    btnXoaXe.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            xuLyXoaXe();
        }
    });
    btnBaoCao.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            xuLyMoBaoCao();
        }
    });
    btnGhiFile.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            xuLyGhiFile();
```

```
            }
        });
        btnKetThuc.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                xuLyKetThuc();
            }
        });

        // Các nút Thêm / Sửa / Xóa Khách hàng mua Xe trên Khung hiển thị chi tiết (Câu 2g)
        btnThemKHMua.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                xuLyThemKhachHangMuaXe();
            }
        });
        btnSuaKHMua.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                xuLySuaKhachHangMuaXe();
            }
        });
        btnXoaKHMua.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                xuLyXoaKhachHangMuaXe();
            }
        });

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                xuLyKetThuc();
            }
        });
    }

    /**
     * Xử lý khi người dùng sửa trực tiếp một ô (cột 1..6) trên bảng Khung hiển thị dữ liệu chi tiết:
     * Tự động kiểm tra tính hợp lệ, cập nhật vào CSDL, tính lại Thành tiền và cập nhật 2 ô Thống kê.
     */
    private void xuLySuaTrucTiepTrenBangChiTiet(int row, int col) {
        if (!repository.kiemTraKetNoi()) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
            return;
        }
        ChiTietMuaXe banGoc = dsChiTietHienTai.get(row);
        String newMaKH = layChuoiO(row, 1);
        String newTenKH = layChuoiO(row, 2);
        String newCmnd = layChuoiO(row, 3);
        String newGioiTinh = layChuoiO(row, 4);
        String newNgayMua = layChuoiO(row, 5);
        String newSoLuongStr = layChuoiO(row, 6);
        String newDiaChi = banGoc.getDiaChi();

        // Nếu người dùng đổi Mã KH sang một khách hàng đã có sẵn thì đồng bộ thông tin khách hàng đó
        if (col == 1) {
            for (int i = 0; i < dsKhachHang.size(); i++) {
                KhachHang kh = dsKhachHang.get(i);
                if (kh.getMaKH().equalsIgnoreCase(newMaKH)) {
                    newTenKH = kh.getTenKH();
                    newCmnd = kh.getCmnd();
```

```
                    newGioiTinh = kh.getGioiTinh();
                    newDiaChi = kh.getDiaChi();
                    break;
                }
            }
        }

        try {
            if (newMaKH.isEmpty() || newTenKH.isEmpty() || newCmnd.isEmpty()
                    || newGioiTinh.isEmpty() || newNgayMua.isEmpty() || newSoLuongStr.isEmpty()) {
                throw new IllegalArgumentException("Ô dữ liệu trên Khung chi tiết không được để trống!");
            }
            if (!newCmnd.matches("[0-9]{8,15}")) {
                throw new IllegalArgumentException("Số CMND phải gồm từ 8 đến 15 chữ số!");
            }
            MuaBanXeRepository.chuyenSangIsoDate(newNgayMua);
            int newSoLuong;
            try {
                newSoLuong = Integer.parseInt(newSoLuongStr);
                if (newSoLuong <= 0) {
                    throw new NumberFormatException();
                }
            } catch (NumberFormatException ex) {
                throw new IllegalArgumentException("Số lượng mua trên bảng phải là số nguyên dương!");
            }

            ChiTietMuaXe capNhat = new ChiTietMuaXe(
                    newMaKH, newTenKH, newCmnd, newGioiTinh, newDiaChi,
                    banGoc.getMaXe(), newNgayMua, newSoLuong, banGoc.getDonGiaXe(), banGoc.getGhiChu()
            );
            repository.suaChiTietMuaXe(banGoc.getMaKH(), banGoc.getMaXe(), banGoc.getNgayMua(), capNhat);
            dsKhachHang = repository.layDanhSachKhachHang();
            napDanhSachMaKHVaoCombo();
            napBangChiTietVaThongKe(banGoc.getMaXe());
            thongBao("Đã tự động lưu thay đổi trên bảng và cập nhật Thành tiền / Thống kê!");
        } catch (Exception ex) {
            thongBao(ex.getMessage());
            napBangChiTietVaThongKe(banGoc.getMaXe());
        }
    }

    private String layChuoiO(int row, int col) {
        Object val = detailTableModel.getValueAt(row, col);
        return val == null ? "" : val.toString().trim();
    }

    /**
     * Lưu ý đầu đề & Câu 2b:
     * Kiểm tra kết nối CSDL đã có kết nối chưa:
     * - Nếu kết nối không được thì đưa dữ liệu trống rỗng lên màn hình cửa sổ và thông báo là kết nối CSDL
không được.
     * - Nếu kết nối thành công thì lấy dữ liệu từ Database Server đưa lên control của Cửa sổ và Khung chi tiết.
     */
    public void kiemTraVaTaiDuLieuBanDau() {
        if (!repository.kiemTraKetNoi()) {
            duaDuLieuTrongRongLenCuaSo();
            thongBao("Kết nối CSDL không được! Đã đưa dữ liệu trống rỗng lên màn hình cửa sổ.");
            SwingUtilities.invokeLater(new Runnable() {
                @Override
                public void run() {
                    txtMaXe.requestFocusInWindow();
                }
```

```
            });
            return;
        }
        taiDuLieuTuCSDLVaHienThi(0);
        // Câu 2b: Khởi tạo cửa sổ chính, con trỏ văn bản đặt vào ô Mã xe
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                txtMaXe.requestFocusInWindow();
            }
        });
    }

    /**
     * Đưa dữ liệu trống rỗng lên màn hình cửa sổ khi kiểm tra kết nối CSDL lúc khởi tạo thất bại.
     */
    public void duaDuLieuTrongRongLenCuaSo() {
        dsXe.clear();
        dsKhachHang.clear();
        dsChiTietHienTai.clear();
        currentIndex = -1;

        isProgrammaticUpdate = true;
        isUpdatingTable = true;
        try {
            txtMaXe.setText("");
            txtMaXe.setBackground(Color.WHITE);
            txtTenXe.setText("");
            txtTenXe.setBackground(Color.WHITE);
            txtDungTich.setText("");
            txtDungTich.setBackground(Color.WHITE);
            txtDonGia.setText("");
            txtDonGia.setBackground(Color.WHITE);
            detailTableModel.setRowCount(0);
            txtTongSoKH.setText("0");
            txtTongThanhTien.setText("0 VNĐ");
            txtChiSo.setText("0/0");
        } finally {
            isUpdatingTable = false;
            isProgrammaticUpdate = false;
        }
        capNhatTrangThaiNutDieuHuong();
    }

    /**
     * Ghi chú 3 (Trang 3):
     * Khi xử lý dữ liệu trên Database Server thì luôn kiểm tra việc kết nối thành công hay không;
     * nếu kết nối dữ liệu không thành công thì tạo ra một dòng dữ liệu ảo, rồi sau đó đưa lên
     * các control chứa dữ liệu thích hợp trên Cửa sổ.
     */
    public boolean kiemTraKetNoiHoacTaoDongDuLieuAo() {
        return kiemTraKetNoiHoacTaoDongDuLieuAo(null);
    }

    public boolean kiemTraKetNoiHoacTaoDongDuLieuAo(Xe customXe) {
        if (repository.kiemTraKetNoi()) {
            return true;
        }
        repository.khoiTaoDongDuLieuAo(customXe);
        try {
            dsXe = repository.layDanhSachXe();
            dsKhachHang = repository.layDanhSachKhachHang();
```

```
            napDanhSachMaKHVaoCombo();
            if (!dsXe.isEmpty()) {
                hienThiXeTaiViTri(0);
            }
        } catch (SQLException ignored) {
        }
        thongBao("Kết nối Database Server không thành công! Đã tạo một dòng dữ liệu ảo và đưa lên các control
trên Cửa sổ.");
        return false;
    }

    public void taiDuLieuTuCSDLVaHienThi(int targetIndex) {
        try {
            dsXe = repository.layDanhSachXe();
            dsKhachHang = repository.layDanhSachKhachHang();
            napDanhSachMaKHVaoCombo();
            if (dsXe.isEmpty()) {
                duaDuLieuTrongRongLenCuaSo();
            } else {
                int idx = Math.max(0, Math.min(targetIndex, dsXe.size() - 1));
                hienThiXeTaiViTri(idx);
            }
        } catch (SQLException ex) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
        }
    }

    private void napDanhSachMaKHVaoCombo() {
        isProgrammaticUpdate = true;
        try {
            cboMaKH.removeAllItems();
            cboTableMaKH.removeAllItems();
            for (int i = 0; i < dsKhachHang.size(); i++) {
                KhachHang kh = dsKhachHang.get(i);
                cboMaKH.addItem(kh.getMaKH());
                cboTableMaKH.addItem(kh.getMaKH());
            }
        } finally {
            isProgrammaticUpdate = false;
        }
    }

    /**
     * Hiển thị bản ghi Xe tại vị trí index lên các control của Cửa sổ và nạp Khung hiển thị dữ liệu chi tiết.
     */
    public void hienThiXeTaiViTri(int index) {
        if (dsXe.isEmpty() || index < 0 || index >= dsXe.size()) {
            duaDuLieuTrongRongLenCuaSo();
            return;
        }
        currentIndex = index;
        Xe xe = dsXe.get(currentIndex);
        isProgrammaticUpdate = true;
        try {
            txtMaXe.setText(xe.getMaXe());
            txtMaXe.setBackground(Color.WHITE);
            txtTenXe.setText(xe.getTenXe());
            txtTenXe.setBackground(Color.WHITE);
            txtDungTich.setText(String.valueOf(xe.getDungTich()));
            txtDungTich.setBackground(Color.WHITE);
            txtDonGia.setText(Xe.dinhDangSo(xe.getDonGia()));
            txtDonGia.setBackground(Color.WHITE);
```

```
            txtChiSo.setText((currentIndex + 1) + "/" + dsXe.size());
        } finally {
            isProgrammaticUpdate = false;
        }

        capNhatTrangThaiNutDieuHuong();
        napBangChiTietVaThongKe(xe.getMaXe());
    }

    /**
     * Câu 2c & 2g: Nạp bảng Khung hiển thị dữ liệu chi tiết (có cột STT đầu tiên)
     * và tính lại 2 control Thống kê (Tổng số khách hàng = n, Tổng thành tiền).
     */
    public void napBangChiTietVaThongKe(String maXe) {
        isUpdatingTable = true;
        try {
            detailTableModel.setRowCount(0);
            dsChiTietHienTai.clear();
            try {
                dsChiTietHienTai = repository.layChiTietMuaTheoXe(maXe);
            } catch (SQLException ex) {
                dsChiTietHienTai = new ArrayList<ChiTietMuaXe>();
            }

            long tongThanhTien = 0L;
            for (int i = 0; i < dsChiTietHienTai.size(); i++) {
                ChiTietMuaXe ct = dsChiTietHienTai.get(i);
                long thanhTien = ct.getThanhTien();
                tongThanhTien += thanhTien;
                detailTableModel.addRow(new Object[]{
                        Integer.valueOf(i + 1),
                        ct.getMaKH(),
                        ct.getTenKH(),
                        ct.getCmnd(),
                        ct.getGioiTinh(),
                        ct.getNgayMua(),
                        Integer.valueOf(ct.getSoLuong()),
                        Xe.dinhDangSo(thanhTien)
                });
            }

            txtTongSoKH.setText(String.valueOf(dsChiTietHienTai.size()));
            txtTongThanhTien.setText(Xe.dinhDangSo(tongThanhTien) + " VNĐ");

            if (!dsChiTietHienTai.isEmpty()) {
                tblChiTiet.setRowSelectionInterval(0, 0);
                dongBoDongChonChiTietLenThanhKH();
            }
        } finally {
            isUpdatingTable = false;
        }
    }

    private void dongBoDongChonChiTietLenThanhKH() {
        int row = tblChiTiet.getSelectedRow();
        if (row >= 0 && row < dsChiTietHienTai.size()) {
            ChiTietMuaXe ct = dsChiTietHienTai.get(row);
            isProgrammaticUpdate = true;
            try {
                cboMaKH.setSelectedItem(ct.getMaKH());
                txtTenKH.setText(ct.getTenKH());
                txtTenKH.setBackground(Color.WHITE);
```

```
            txtCMND.setText(ct.getCmnd());
            txtCMND.setBackground(Color.WHITE);
            cboGioiTinh.setSelectedItem(ct.getGioiTinh());
            txtDiaChiKH.setText(ct.getDiaChi());
            txtNgayMua.setText(ct.getNgayMua());
            txtNgayMua.setBackground(Color.WHITE);
            txtSoLuongMua.setText(String.valueOf(ct.getSoLuong()));
            txtSoLuongMua.setBackground(Color.WHITE);
            txtGhiChuMua.setText(ct.getGhiChu());
        } finally {
            isProgrammaticUpdate = false;
        }
    }
}

private void capNhatTrangThaiNutDieuHuong() {
    int n = dsXe.size();
    btnVeDau.setEnabled(n > 0 && currentIndex > 0);
    btnVeTruoc.setEnabled(n > 0 && currentIndex > 0);
    btnVeSau.setEnabled(n > 0 && currentIndex >= 0 && currentIndex < n - 1);
    btnVeCuoi.setEnabled(n > 0 && currentIndex >= 0 && currentIndex < n - 1);
}

public void diChuyenDenXe(int newIndex) {
    if (newIndex >= 0 && newIndex < dsXe.size()) {
        hienThiXeTaiViTri(newIndex);
    }
}

/**
 * Kiểm tra ràng buộc các ô nhập Xe trên Cửa sổ chính (Câu 2b & 2e).
 */
public Xe kiemTraVaDocThongTinXeTuForm() {
    String maXe = txtMaXe.getText().trim();
    if (maXe.isEmpty()) {
        txtMaXe.requestFocusInWindow();
        throw new IllegalArgumentException("Mã xe không được phép để trống (rỗng)!");
    }
    String tenXe = txtTenXe.getText().trim();
    if (tenXe.isEmpty()) {
        txtTenXe.requestFocusInWindow();
        throw new IllegalArgumentException("Tên xe không được phép để trống (rỗng)!");
    }
    String dungTichStr = txtDungTich.getText().trim();
    if (dungTichStr.isEmpty()) {
        txtDungTich.requestFocusInWindow();
        throw new IllegalArgumentException("Dung tích không được phép để trống (rỗng)!");
    }
    int dungTich;
    try {
        dungTich = Integer.parseInt(dungTichStr);
        if (dungTich <= 0) {
            throw new NumberFormatException();
        }
    } catch (NumberFormatException ex) {
        txtDungTich.requestFocusInWindow();
        throw new IllegalArgumentException("Dung tích phải là số nguyên dương!");
    }

    String donGiaStr = txtDonGia.getText().trim();
    if (donGiaStr.isEmpty()) {
        txtDonGia.requestFocusInWindow();
```

```
            throw new IllegalArgumentException("Đơn giá không được phép để trống (rỗng)!");
        }
        long donGia;
        try {
            donGia = Xe.parseDonGia(donGiaStr);
            if (donGia <= 0) {
                throw new NumberFormatException();
            }
        } catch (NumberFormatException ex) {
            txtDonGia.requestFocusInWindow();
            throw new IllegalArgumentException("Đơn giá phải là số nguyên dương!");
        }
        return new Xe(maXe, tenXe, dungTich, donGia);
    }

    /**
     * Câu 2e: Btn Thêm xe:
     * - Hỏi xác nhận Yes/No.
     * - Kiểm tra kết nối CSDL (nếu mất kết nối thì tạo dòng dữ liệu ảo theo Ghi chú 3).
     * - Kiểm tra thông tin ràng buộc không rỗng.
     * - Kiểm tra Mã xe đã tồn tại trong table Xe chưa; nếu tồn tại thì thông báo lỗi:
     *   "Mã xe đã tồn tại, vui lòng nhập lại!"
     * - Ngược lại lưu thông tin Xe vào Database Server, sau đó cập nhật các control Thống kê
     *   và Khung hiển thị dữ liệu chi tiết.
     */
    public void xuLyThemXe() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn thực thi chức năng Thêm Xe hay không?")) {
            return;
        }
        if (!repository.kiemTraKetNoi()) {
            Xe customXe = null;
            try {
                customXe = kiemTraVaDocThongTinXeTuForm();
            } catch (IllegalArgumentException ignored) {
            }
            kiemTraKetNoiHoacTaoDongDuLieuAo(customXe);
            return;
        }
        Xe xeMoi;
        try {
            xeMoi = kiemTraVaDocThongTinXeTuForm();
        } catch (IllegalArgumentException ex) {
            thongBao(ex.getMessage());
            return;
        }

        try {
            if (repository.tonTaiMaXe(xeMoi.getMaXe())) {
                txtMaXe.requestFocusInWindow();
                txtMaXe.selectAll();
                thongBao("Mã xe đã tồn tại, vui lòng nhập lại!");
                return;
            }
            repository.themXe(xeMoi);

            dsXe = repository.layDanhSachXe();
            int newIdx = dsXe.size() - 1;
            for (int i = 0; i < dsXe.size(); i++) {
                if (dsXe.get(i).getMaXe().equalsIgnoreCase(xeMoi.getMaXe())) {
                    newIdx = i;
                    break;
                }
```

```
            }
            hienThiXeTaiViTri(newIdx);
            thongBao("Thêm Xe '" + xeMoi.getMaXe() + "' vào Database Server thành công!");
        } catch (SQLException ex) {
            thongBao(ex.getMessage());
        }
    }

    /**
     * Câu 2f: Btn Xóa Xe:
     * - Hỏi xác nhận Yes/No (nêu rõ xóa Xe kèm lịch sử mua của Xe đó).
     * - Xóa dòng dữ liệu hiện thời trên Cửa sổ (thông tin Xe) và cập nhật vào Database Server.
     * - Sau đó đưa dữ liệu của Xe kế tiếp lên Cửa sổ (cập nhật lại các control Thống kê).
     */
    public void xuLyXoaXe() {
        String maXeHienTai = txtMaXe.getText().trim();
        String msgXacNhan = maXeHienTai.isEmpty()
                ? "Bạn có chắc chắn muốn xóa Xe hiện thời khỏi Database Server hay không?"
                : "Bạn có chắc chắn muốn xóa Xe '" + maXeHienTai
                  + "' (bao gồm lịch sử khách hàng mua xe này) khỏi Database Server hay không?";
        if (!xacNhanThucThi(msgXacNhan)) {
            return;
        }
        if (!repository.kiemTraKetNoi()) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
            return;
        }
        if (maXeHienTai.isEmpty() || dsXe.isEmpty()) {
            thongBao("Không có bản ghi Xe hiện thời để xóa!");
            return;
        }
        int viTriKeTiep = currentIndex;
        try {
            repository.xoaXe(maXeHienTai);
            dsXe = repository.layDanhSachXe();
            if (dsXe.isEmpty()) {
                duaDuLieuTrongRongLenCuaSo();
            } else {
                if (viTriKeTiep >= dsXe.size()) {
                    viTriKeTiep = dsXe.size() - 1;
                }
                hienThiXeTaiViTri(viTriKeTiep);
            }
            thongBao("Đã xóa Xe '" + maXeHienTai + "' và đưa dữ liệu Xe kế tiếp lên Cửa sổ.");
        } catch (SQLException ex) {
            thongBao("Lỗi khi xóa Xe: " + ex.getMessage());
        }
    }

    /**
     * Câu 2g: Thêm / Sửa / Xóa thông tin Khách hàng đã mua Xe đã chọn trên Khung hiển thị dữ liệu chi tiết.
     */
    public void xuLyThemKhachHangMuaXe() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn thêm Khách hàng mua Xe này vào CSDL hay không?")) {
            return;
        }
        if (!repository.kiemTraKetNoi()) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
            return;
        }
        if (currentIndex < 0 || currentIndex >= dsXe.size()) {
            thongBao("Vui lòng chọn một Xe trước khi thêm Khách hàng mua!");
```

```
            return;
        }
        try {
            ChiTietMuaXe ct = docChiTietMuaTuThanhKH();
            repository.themChiTietMuaXe(ct);
            dsKhachHang = repository.layDanhSachKhachHang();
            napDanhSachMaKHVaoCombo();
            napBangChiTietVaThongKe(dsXe.get(currentIndex).getMaXe());
            thongBao("Đã thêm Khách hàng '" + ct.getMaKH() + "' mua xe '" + ct.getMaXe() + "' thành công!");
        } catch (Exception ex) {
            thongBao(ex.getMessage());
        }
    }

    public void xuLySuaKhachHangMuaXe() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn cập nhật (Sửa) dòng Khách hàng mua Xe đang chọn hay không?"))
{
            return;
        }
        if (!repository.kiemTraKetNoi()) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
            return;
        }
        int row = tblChiTiet.getSelectedRow();
        if (row < 0 || row >= dsChiTietHienTai.size()) {
            thongBao("Vui lòng chọn một dòng Khách hàng trên Khung hiển thị chi tiết để sửa!");
            return;
        }
        ChiTietMuaXe banGoc = dsChiTietHienTai.get(row);
        try {
            ChiTietMuaXe capNhat = docChiTietMuaTuThanhKH();
            repository.suaChiTietMuaXe(banGoc.getMaKH(), banGoc.getMaXe(), banGoc.getNgayMua(), capNhat);
            dsKhachHang = repository.layDanhSachKhachHang();
            napDanhSachMaKHVaoCombo();
            napBangChiTietVaThongKe(banGoc.getMaXe());
            thongBao("Đã cập nhật thông tin Khách hàng mua xe thành công!");
        } catch (Exception ex) {
            thongBao(ex.getMessage());
        }
    }

    public void xuLyXoaKhachHangMuaXe() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn xóa dòng Khách hàng mua Xe đang chọn hay không?")) {
            return;
        }
        if (!repository.kiemTraKetNoi()) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
            return;
        }
        int row = tblChiTiet.getSelectedRow();
        if (row < 0 || row >= dsChiTietHienTai.size()) {
            thongBao("Vui lòng chọn một dòng Khách hàng trên Khung hiển thị chi tiết để xóa!");
            return;
        }
        ChiTietMuaXe ct = dsChiTietHienTai.get(row);
        try {
            repository.xoaChiTietMuaXe(ct.getMaKH(), ct.getMaXe(), ct.getNgayMua());
            napBangChiTietVaThongKe(ct.getMaXe());
            thongBao("Đã xóa Khách hàng '" + ct.getMaKH() + "' khỏi danh sách mua xe '" + ct.getMaXe() + "'.");
        } catch (Exception ex) {
            thongBao(ex.getMessage());
        }
```

```
    }

    private ChiTietMuaXe docChiTietMuaTuThanhKH() throws SQLException {
        Xe xe = dsXe.get(currentIndex);
        String maKH = cboMaKH.getSelectedItem() == null ? "" : cboMaKH.getSelectedItem().toString().trim();
        String tenKH = txtTenKH.getText().trim();
        String cmnd = txtCMND.getText().trim();
        String gioiTinh = cboGioiTinh.getSelectedItem() == null ? "Nam" :
cboGioiTinh.getSelectedItem().toString().trim();
        String diaChi = txtDiaChiKH.getText().trim();
        String ngayMua = txtNgayMua.getText().trim();
        String slStr = txtSoLuongMua.getText().trim();

        if (maKH.isEmpty() || tenKH.isEmpty() || cmnd.isEmpty() || ngayMua.isEmpty() || slStr.isEmpty()) {
            throw new IllegalArgumentException("Thông tin Khách hàng mua xe (Mã KH, Tên KH, CMND, Ngày mua, Số
lượng) không được để trống!");
        }
        if (!cmnd.matches("[0-9]{8,15}")) {
            throw new IllegalArgumentException("Số CMND phải gồm từ 8 đến 15 chữ số!");
        }
        if (diaChi.isEmpty()) {
            for (int i = 0; i < dsKhachHang.size(); i++) {
                if (dsKhachHang.get(i).getMaKH().equalsIgnoreCase(maKH)) {
                    diaChi = dsKhachHang.get(i).getDiaChi();
                    break;
                }
            }
            if (diaChi.isEmpty()) {
                diaChi = "TP.HCM";
            }
        }
        MuaBanXeRepository.chuyenSangIsoDate(ngayMua); // Kiểm tra định dạng dd/MM/yyyy
        int soLuong;
        try {
            soLuong = Integer.parseInt(slStr);
            if (soLuong <= 0) {
                throw new NumberFormatException();
            }
        } catch (NumberFormatException ex) {
            throw new IllegalArgumentException("Số lượng mua phải là số nguyên dương!");
        }
        return new ChiTietMuaXe(maKH, tenKH, cmnd, gioiTinh, diaChi,
                xe.getMaXe(), ngayMua, soLuong, xe.getDonGia(), txtGhiChuMua.getText().trim());
    }

    /**
     * Câu 2h: Btn Báo cáo:
     * Xuất hiện Cửa sổ Báo cáo có Combobox "Mã xe", Txtbox "Ngày mua", và Btn "Thực hiện báo cáo".
     */
    public void xuLyMoBaoCao() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn mở Cửa sổ Báo cáo hay không?")) {
            return;
        }
        String maXeHienTai = txtMaXe.getText().trim();
        lastReportDialog = new BaoCaoMuaXeDialog(this, repository, maXeHienTai, isTestMode);
        lastReportDialog.setVisible(true);
    }

    /**
     * Câu 2i: Btn Ghi file:
     * Tạo một file .txt trên thiết bị lưu trữ dữ liệu để lưu thông tin trên Khung hiển thị dữ liệu chi tiết
hiện thời.
```

```
     */
    public void xuLyGhiFile() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn ghi thông tin Khung hiển thị dữ liệu chi tiết hiện thời ra
file txt hay không?")) {
            return;
        }
        File targetFile;
        if (isTestMode) {
            targetFile = new File("ThongKeKhachHangMuaXe_" + txtMaXe.getText().trim() + ".txt");
        } else {
            JFileChooser chooser = new JFileChooser();
            chooser.setDialogTitle("Lưu Khung hiển thị dữ liệu chi tiết ra file TXT");
            chooser.setSelectedFile(new File("KhachHangMuaXe_" + txtMaXe.getText().trim() + ".txt"));
            if (chooser.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) {
                return;
            }
            targetFile = chooser.getSelectedFile();
        }
        luuChiTietRaFileTxt(targetFile);
    }

    public void luuChiTietRaFileTxt(File file) {
        PrintWriter out = null;
        try {
            out = new PrintWriter(file, StandardCharsets.UTF_8.name());

out.println("=============================================================================================");
            out.println("                  TRUNG TÂM XE GẮN MÁY NVT - KHUNG HIỂN THỊ DỮ LIỆU CHI TIẾT
");

out.println("=============================================================================================");
            out.println("Mã xe      : " + txtMaXe.getText().trim());
            out.println("Tên xe     : " + txtTenXe.getText().trim());
            out.println("Dung tích  : " + txtDungTich.getText().trim());
            out.println("Đơn giá    : " + txtDonGia.getText().trim() + " VNĐ");
            out.println("---------------------------------------------------------------------------------------
-------");
            out.printf("%-5s | %-8s | %-22s | %-12s | %-9s | %-12s | %-8s | %-15s%n",
                    "STT", "Mã KH", "Tên khách hàng", "Số CMND", "Giới tính", "Ngày mua", "Số lượng", "Thành
tiền");
            out.println("---------------------------------------------------------------------------------------
-------");
            for (int i = 0; i < dsChiTietHienTai.size(); i++) {
                ChiTietMuaXe ct = dsChiTietHienTai.get(i);
                out.printf("%-5d | %-8s | %-22s | %-12s | %-9s | %-12s | %-8d | %-15s%n",
                        Integer.valueOf(i + 1),
                        ct.getMaKH(),
                        ct.getTenKH(),
                        ct.getCmnd(),
                        ct.getGioiTinh(),
                        ct.getNgayMua(),
                        Integer.valueOf(ct.getSoLuong()),
                        Xe.dinhDangSo(ct.getThanhTien()));
            }

out.println("=============================================================================================");
            out.println("Tổng số khách hàng : " + txtTongSoKH.getText().trim());
            out.println("Tổng thành tiền    : " + txtTongThanhTien.getText().trim());

out.println("=============================================================================================");
            out.flush();
            thongBao("Đã ghi dữ liệu Khung chi tiết ra file thành công:\n" + file.getAbsolutePath());
```

```
        } catch (Exception ex) {
            thongBao("Lỗi khi ghi file txt: " + ex.getMessage());
        } finally {
            if (out != null) {
                out.close();
            }
        }
    }

    /**
     * Nút Kết thúc: Hỏi xác nhận Yes/No trước khi đóng chương trình.
     */
    public void xuLyKetThuc() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn kết thúc và thoát chương trình hay không?")) {
            return;
        }
        if (lastReportDialog != null && lastReportDialog.isDisplayable()) {
            lastReportDialog.dispose();
        }
        dispose();
        if (!isTestMode) {
            System.exit(0);
        }
    }

    /**
     * Ghi chú 1 (Trang 3):
     * Tất cả các btn trên các Cửa sổ thì đều có xuất hiện thông báo hỏi người dùng có chắc chắn
     * thực thi tiếp hay không? Yes: đồng ý, No: không.
     */
    private boolean xacNhanThucThi(String message) {
        if (isTestMode) {
            return true;
        }
        int choice = JOptionPane.showConfirmDialog(
                this,
                message,
                "Xác nhận thực thi",
                JOptionPane.YES_NO_OPTION,
                JOptionPane.QUESTION_MESSAGE
        );
        return choice == JOptionPane.YES_OPTION;
    }

    private void thongBao(String message) {
        this.lastAlertMessage = message;
        if (!isTestMode) {
            JOptionPane.showMessageDialog(this, message, "Thông báo", JOptionPane.INFORMATION_MESSAGE);
        }
    }

    // Các phương thức Getter chuẩn OOP phục vụ kiểm thử tự động (không dùng Reflection)
    public String getLastAlertMessage() { return lastAlertMessage; }
    public BaoCaoMuaXeDialog getLastReportDialog() { return lastReportDialog; }
    public List<Xe> getDsXe() { return dsXe; }
    public List<KhachHang> getDsKhachHang() { return dsKhachHang; }
    public List<ChiTietMuaXe> getDsChiTietHienTai() { return dsChiTietHienTai; }
    public int getCurrentIndex() { return currentIndex; }

    public static void khoiChayGiaoDien(String[] args) {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
```

```
        } catch (Exception ignored) {
        }
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new QuanLyMuaBanXeGUI().setVisible(true);
            }
        });
    }
}

/**
 * Cửa sổ Báo cáo (Câu 2h):
 * - Chứa Combobox "Mã xe", Txtbox "Ngày mua", và Btn "Thực hiện báo cáo".
 * - Phát sinh báo cáo theo chuẩn giao diện Crystal Report trong đề thi:
 *   "Cửa hàng Mô tô PKL - THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE"
 *   với bảng 5 cột: STT | Mã khách hàng | Ngày mua | Số lượng | Ghi chú.
 */
@SuppressWarnings("serial")
class BaoCaoMuaXeDialog extends JDialog {
    private static final long serialVersionUID = 1L;

    private final MuaBanXeRepository repository;
    private final boolean autoConfirmMode;

    private JComboBox<String> cboMaXe;
    private JTextField txtNgayMua;
    private JButton btnThucHienBaoCao;
    private JButton btnDongBaoCao;

    private JLabel lblValMaXe;
    private JLabel lblValTenXe;
    private JLabel lblValDonGia;
    private JLabel lblValDungTich;
    private JLabel lblValSoLuongKH;
    private JLabel lblNgayLap;

    private DefaultTableModel reportTableModel;
    private JTable tblBaoCao;
    private List<Xe> danhSachXe = new ArrayList<Xe>();
    private String lastErrorMessage = "";

    public BaoCaoMuaXeDialog(JFrame parent, MuaBanXeRepository repository, String initialMaXe, boolean
autoConfirmMode) {
        super(parent, "Cửa sổ Báo cáo - Thống kê khách hàng đã mua xe", false);
        this.repository = repository;
        this.autoConfirmMode = autoConfirmMode;
        khoiTaoGiaoDien();
        napDanhSachMaXe(initialMaXe);
        thucHienPhatSinhBaoCao(false);
    }

    private void khoiTaoGiaoDien() {
        setSize(760, 560);
        setLocationRelativeTo(getOwner());
        setLayout(new BorderLayout(8, 8));

        // 1. Thanh tiêu chí lọc ở phía Bắc (Combobox "Mã xe", Txtbox "Ngày mua", Btn "Thực hiện báo cáo")
        JPanel pnlFilter = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 8));
        pnlFilter.setBackground(new Color(238, 242, 248));
        pnlFilter.setBorder(BorderFactory.createTitledBorder(
                new LineBorder(new Color(140, 160, 190), 1),
```

```
                "Tiêu chí lọc báo cáo (Tổ hợp lựa chọn)"
        ));

        JLabel lblLocMaXe = new JLabel("Mã xe:");
        lblLocMaXe.setFont(new Font("Tahoma", Font.BOLD, 12));
        cboMaXe = new JComboBox<String>();
        cboMaXe.setPreferredSize(new Dimension(130, 26));
        cboMaXe.setFont(new Font("Tahoma", Font.PLAIN, 12));

        JLabel lblLocNgayMua = new JLabel("Ngày mua (dd/mm/yyyy):");
        lblLocNgayMua.setFont(new Font("Tahoma", Font.BOLD, 12));
        txtNgayMua = new JTextField(12);
        txtNgayMua.setFont(new Font("Tahoma", Font.PLAIN, 12));
        txtNgayMua.setToolTipText("Nhập ngày mua cụ thể theo định dạng dd/mm/yyyy (ví dụ 28/10/2012) hoặc để
trống để xem tất cả ngày mua");

        btnThucHienBaoCao = new JButton("Thực hiện báo cáo");
        btnThucHienBaoCao.setFont(new Font("Tahoma", Font.BOLD, 12));
        btnThucHienBaoCao.setForeground(new Color(18, 53, 126));

        btnDongBaoCao = new JButton("Đóng");
        btnDongBaoCao.setFont(new Font("Tahoma", Font.BOLD, 12));

        pnlFilter.add(lblLocMaXe);
        pnlFilter.add(cboMaXe);
        pnlFilter.add(lblLocNgayMua);
        pnlFilter.add(txtNgayMua);
        pnlFilter.add(btnThucHienBaoCao);
        pnlFilter.add(btnDongBaoCao);

        add(pnlFilter, BorderLayout.NORTH);

        // 2. Vùng hiển thị Báo cáo chuẩn Crystal Report ở Trung tâm
        JPanel pnlCrystalReport = new JPanel(new BorderLayout(8, 8));
        pnlCrystalReport.setBackground(Color.WHITE);
        pnlCrystalReport.setBorder(new CompoundBorder(
                new EmptyBorder(4, 12, 12, 12),
                new CompoundBorder(new LineBorder(new Color(180, 180, 180), 1), new EmptyBorder(14, 22, 14, 22))
        ));

        // 2.1. Phần đầu báo cáo (Header + Thông tin Xe)
        JPanel pnlReportHeader = new JPanel(new BorderLayout(4, 8));
        pnlReportHeader.setBackground(Color.WHITE);

        JPanel pnlTitles = new JPanel(new GridLayout(2, 1, 0, 4));
        pnlTitles.setBackground(Color.WHITE);
        JLabel lblStore = new JLabel("Cửa hàng Mô tô PKL", SwingConstants.LEFT);
        lblStore.setFont(new Font("Times New Roman", Font.BOLD | Font.ITALIC, 15));
        JLabel lblMainReportTitle = new JLabel("THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE", SwingConstants.CENTER);
        lblMainReportTitle.setFont(new Font("Times New Roman", Font.BOLD, 24));
        pnlTitles.add(lblStore);
        pnlTitles.add(lblMainReportTitle);
        pnlReportHeader.add(pnlTitles, BorderLayout.NORTH);

        // Thông tin Xe được chọn: Mã xe, Tên xe, Đơn Giá, Dung tích
        JPanel pnlXeInfo = new JPanel(new GridBagLayout());
        pnlXeInfo.setBackground(Color.WHITE);
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(4, 8, 4, 8);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        Font lblBold = new Font("Times New Roman", Font.BOLD, 15);
```

```
        Font valFont = new Font("Times New Roman", Font.BOLD, 15);
        Color valBlue = new Color(25, 55, 150);

        lblValMaXe = taoNhanGiaTriBaoCao(valFont, valBlue);
        lblValTenXe = taoNhanGiaTriBaoCao(valFont, valBlue);
        lblValDonGia = taoNhanGiaTriBaoCao(valFont, valBlue);
        lblValDungTich = taoNhanGiaTriBaoCao(valFont, valBlue);

        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0;
        pnlXeInfo.add(taoNhanTieuDe("Mã xe:", lblBold), gbc);
        gbc.gridx = 1; gbc.weightx = 0.45;
        pnlXeInfo.add(lblValMaXe, gbc);

        gbc.gridx = 2; gbc.weightx = 0;
        pnlXeInfo.add(taoNhanTieuDe("Tên xe:", lblBold), gbc);
        gbc.gridx = 3; gbc.weightx = 0.55;
        pnlXeInfo.add(lblValTenXe, gbc);

        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0;
        pnlXeInfo.add(taoNhanTieuDe("Đơn Giá:", lblBold), gbc);
        gbc.gridx = 1; gbc.weightx = 0.45;
        pnlXeInfo.add(lblValDonGia, gbc);

        gbc.gridx = 2; gbc.weightx = 0;
        pnlXeInfo.add(taoNhanTieuDe("Dung tích:", lblBold), gbc);
        gbc.gridx = 3; gbc.weightx = 0.55;
        pnlXeInfo.add(lblValDungTich, gbc);

        pnlReportHeader.add(pnlXeInfo, BorderLayout.CENTER);
        pnlCrystalReport.add(pnlReportHeader, BorderLayout.NORTH);

        // 2.2. Bảng báo cáo 5 cột: STT | Mã khách hàng | Ngày mua | Số lượng | Ghi chú
        reportTableModel = new DefaultTableModel(
                new String[]{"STT", "Mã khách hàng", "Ngày mua", "Số lượng", "Ghi chú"}, 0
        ) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return false;
            }
        };
        tblBaoCao = new JTable(reportTableModel);
        tblBaoCao.setRowHeight(25);
        tblBaoCao.setFont(new Font("Times New Roman", Font.PLAIN, 15));
        tblBaoCao.setGridColor(new Color(180, 120, 140));
        tblBaoCao.setShowGrid(true);

        JTableHeader header = tblBaoCao.getTableHeader();
        header.setPreferredSize(new Dimension(100, 28));
        header.setDefaultRenderer(new DefaultTableCellRenderer() {
            @Override
            public Component getTableCellRendererComponent(JTable table, Object value,
                                                           boolean isSelected, boolean hasFocus, int row, int
column) {
                JLabel lbl = (JLabel) super.getTableCellRendererComponent(table, value, isSelected, hasFocus,
row, column);
                lbl.setBackground(new Color(82, 182, 72));
                lbl.setForeground(Color.WHITE);
                lbl.setFont(new Font("Times New Roman", Font.BOLD, 15));
                lbl.setHorizontalAlignment(SwingConstants.CENTER);
                lbl.setBorder(new LineBorder(new Color(160, 110, 130), 1));
                lbl.setOpaque(true);
                return lbl;
```

```
        }
    });

    DefaultTableCellRenderer centerRenderer = new DefaultTableCellRenderer();
    centerRenderer.setHorizontalAlignment(SwingConstants.CENTER);
    tblBaoCao.getColumnModel().getColumn(0).setCellRenderer(centerRenderer);
    tblBaoCao.getColumnModel().getColumn(1).setCellRenderer(centerRenderer);
    tblBaoCao.getColumnModel().getColumn(2).setCellRenderer(centerRenderer);
    tblBaoCao.getColumnModel().getColumn(3).setCellRenderer(centerRenderer);

    tblBaoCao.getColumnModel().getColumn(0).setPreferredWidth(50);
    tblBaoCao.getColumnModel().getColumn(1).setPreferredWidth(140);
    tblBaoCao.getColumnModel().getColumn(2).setPreferredWidth(120);
    tblBaoCao.getColumnModel().getColumn(3).setPreferredWidth(90);
    tblBaoCao.getColumnModel().getColumn(4).setPreferredWidth(180);

    JScrollPane scrollReport = new JScrollPane(tblBaoCao);
    scrollReport.getViewport().setBackground(Color.WHITE);
    pnlCrystalReport.add(scrollReport, BorderLayout.CENTER);

    // 2.3. Chân trang Báo cáo (Số lượng khách hàng + Ngày tháng + Người lập)
    JPanel pnlFooter = new JPanel(new BorderLayout(4, 6));
    pnlFooter.setBackground(Color.WHITE);

    JPanel pnlCount = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 2));
    pnlCount.setBackground(Color.WHITE);
    JLabel lblCountTitle = new JLabel("Số lượng khách hàng:");
    lblCountTitle.setFont(lblBold);
    lblValSoLuongKH = new JLabel("0");
    lblValSoLuongKH.setFont(valFont);
    lblValSoLuongKH.setForeground(valBlue);
    lblValSoLuongKH.setBorder(new MatteBorder(0, 0, 1, 0, Color.DARK_GRAY));
    pnlCount.add(lblCountTitle);
    pnlCount.add(lblValSoLuongKH);
    pnlFooter.add(pnlCount, BorderLayout.NORTH);

    JPanel pnlSign = new JPanel(new GridLayout(4, 1, 0, 2));
    pnlSign.setBackground(Color.WHITE);
    pnlSign.setPreferredSize(new Dimension(280, 90));
    String ngayHienTai = new SimpleDateFormat("'TP.HCM, ngày' dd 'tháng' MM 'năm' yyyy").format(new Date());
    lblNgayLap = new JLabel(ngayHienTai, SwingConstants.CENTER);
    lblNgayLap.setFont(new Font("Times New Roman", Font.BOLD, 14));
    JLabel lblNguoiLap = new JLabel("Người lập", SwingConstants.CENTER);
    lblNguoiLap.setFont(new Font("Times New Roman", Font.BOLD, 14));
    JLabel lblBlank = new JLabel("", SwingConstants.CENTER);
    JLabel lblTenNguoiLap = new JLabel("Trần Mạnh Hùng (AT21N0129)", SwingConstants.CENTER);
    lblTenNguoiLap.setFont(new Font("Times New Roman", Font.BOLD, 14));
    pnlSign.add(lblNgayLap);
    pnlSign.add(lblNguoiLap);
    pnlSign.add(lblBlank);
    pnlSign.add(lblTenNguoiLap);

    JPanel pnlSignWrap = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    pnlSignWrap.setBackground(Color.WHITE);
    pnlSignWrap.add(pnlSign);
    pnlFooter.add(pnlSignWrap, BorderLayout.CENTER);

    pnlCrystalReport.add(pnlFooter, BorderLayout.SOUTH);
    add(pnlCrystalReport, BorderLayout.CENTER);

    // Sự kiện các nút trên cửa sổ Báo cáo (đều hỏi Yes/No theo Ghi chú 1)
    btnThucHienBaoCao.addActionListener(new ActionListener() {
```

```
            @Override
            public void actionPerformed(ActionEvent e) {
                if (xacNhan("Bạn có chắc chắn muốn thực hiện phát sinh báo cáo theo tiêu chí đã chọn hay
không?")) {
                    thucHienPhatSinhBaoCao(true);
                }
            }
        });

        btnDongBaoCao.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (xacNhan("Bạn có chắc chắn muốn đóng Cửa sổ Báo cáo hay không?")) {
                    dispose();
                }
            }
        });
    }

    private static JLabel taoNhanTieuDe(String text, Font font) {
        JLabel lbl = new JLabel(text);
        lbl.setFont(font);
        return lbl;
    }

    private static JLabel taoNhanGiaTriBaoCao(Font font, Color color) {
        JLabel lbl = new JLabel(" ");
        lbl.setFont(font);
        lbl.setForeground(color);
        lbl.setBorder(new MatteBorder(0, 0, 1, 0, Color.GRAY));
        return lbl;
    }

    private boolean xacNhan(String message) {
        if (autoConfirmMode) {
            return true;
        }
        int ans = JOptionPane.showConfirmDialog(this, message, "Xác nhận thực thi",
                JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE);
        return ans == JOptionPane.YES_OPTION;
    }

    public void napDanhSachMaXe(String initialMaXe) {
        cboMaXe.removeAllItems();
        try {
            danhSachXe = repository.layDanhSachXe();
        } catch (SQLException e) {
            repository.khoiTaoDongDuLieuAo();
            try {
                danhSachXe = repository.layDanhSachXe();
            } catch (SQLException ignored) {
            }
        }
        for (int i = 0; i < danhSachXe.size(); i++) {
            cboMaXe.addItem(danhSachXe.get(i).getMaXe());
        }
        cboMaXe.addItem("-- Tất cả --");
        if (initialMaXe != null && !initialMaXe.trim().isEmpty()) {
            cboMaXe.setSelectedItem(initialMaXe.trim());
        } else if (cboMaXe.getItemCount() > 0) {
            cboMaXe.setSelectedIndex(0);
        }
```

```
    }

    public void thucHienPhatSinhBaoCao(boolean showCheckConnectionAlert) {
        lastErrorMessage = "";
        if (!repository.kiemTraKetNoi()) {
            if (showCheckConnectionAlert && !autoConfirmMode) {
                JOptionPane.showMessageDialog(this,
                        "Kết nối Database Server không thành công!\nHệ thống sử dụng dòng dữ liệu ảo để hiển thị
báo cáo.",
                        "Cảnh báo kết nối CSDL", JOptionPane.WARNING_MESSAGE);
            }
            repository.khoiTaoDongDuLieuAo();
        }
        String selectedMaXe = cboMaXe.getSelectedItem() == null ? "" :
cboMaXe.getSelectedItem().toString().trim();
        String ngayMuaFilter = txtNgayMua.getText().trim();

        Xe xeInfo = null;
        for (int i = 0; i < danhSachXe.size(); i++) {
            Xe x = danhSachXe.get(i);
            if (x.getMaXe().equalsIgnoreCase(selectedMaXe)) {
                xeInfo = x;
                break;
            }
        }
        if (xeInfo != null) {
            lblValMaXe.setText(xeInfo.getMaXe());
            lblValTenXe.setText(xeInfo.getTenXe());
            lblValDonGia.setText(Xe.dinhDangSo(xeInfo.getDonGia()) + " VNĐ");
            lblValDungTich.setText(String.valueOf(xeInfo.getDungTich()) + " cc");
        } else {
            lblValMaXe.setText("Tất cả mã xe");
            lblValTenXe.setText("Tất cả dòng xe");
            lblValDonGia.setText("---");
            lblValDungTich.setText("---");
        }

        reportTableModel.setRowCount(0);
        try {
            List<ChiTietMuaXe> rows = repository.locBaoCaoMuaXe(selectedMaXe, ngayMuaFilter);
            for (int i = 0; i < rows.size(); i++) {
                ChiTietMuaXe ct = rows.get(i);
                reportTableModel.addRow(new Object[]{
                        Integer.valueOf(i + 1),
                        ct.getMaKH(),
                        ct.getNgayMua(),
                        Integer.valueOf(ct.getSoLuong()),
                        ct.getGhiChu()
                });
            }
            lblValSoLuongKH.setText("  " + rows.size() + "  ");
        } catch (SQLException ex) {
            lastErrorMessage = ex.getMessage();
            lblValSoLuongKH.setText("  0  ");
            if (showCheckConnectionAlert && !autoConfirmMode) {
                JOptionPane.showMessageDialog(this, ex.getMessage(),
                        "Lỗi tiêu chí lọc báo cáo", JOptionPane.WARNING_MESSAGE);
            }
        }
    }

    // Các phương thức Getter chuẩn OOP phục vụ kiểm thử tự động
```

```
    public JComboBox<String> getCboMaXe() { return cboMaXe; }
    public JTextField getTxtNgayMua() { return txtNgayMua; }
    public JButton getBtnThucHienBaoCao() { return btnThucHienBaoCao; }
    public JTable getTblBaoCao() { return tblBaoCao; }
    public JLabel getLblValMaXe() { return lblValMaXe; }
    public JLabel getLblValTenXe() { return lblValTenXe; }
    public JLabel getLblValSoLuongKH() { return lblValSoLuongKH; }
    public String getLastErrorMessage() { return lastErrorMessage; }
}

/**
 * Tầng truy xuất dữ liệu CSDL SQL Server QLMuaBanXe.
 *
 * Đáp ứng đầy đủ yêu cầu đề thi và tính khả chuyển đa môi trường:
 * 1. Lưu ý đầu đề: Kiểm tra kết nối CSDL đã có kết nối chưa; nếu kết nối không được
 *    thì thông báo và đưa dữ liệu trống rỗng lên màn hình.
 * 2. Ghi chú cuối đề: Khi xử lý dữ liệu trên Database Server thì luôn kiểm tra việc
 *    kết nối thành công hay không; nếu kết nối không thành công thì tạo ra một dòng
 *    dữ liệu ảo, rồi sau đó đưa lên các control chứa dữ liệu thích hợp trên Cửa sổ.
 * 3. Hỗ trợ cấu hình linh hoạt qua tệp config.properties / db.properties, System properties,
 *    biến môi trường, tự động dò tìm nhiều SQL Server instance phổ biến (.\SQL2025DEV,
 *    localhost, ., .\SQLEXPRESS, (local)), và hỗ trợ chế độ giả lập CSDL (Simulated DB Mode)
 *    trên các máy chấm bài không cài đặt SQL Server.
 */
class MuaBanXeRepository {
    private static final String DEFAULT_HOST = "127.0.0.1";
    private static final String DEFAULT_PORT = "1433";
    private static final String DEFAULT_DB_NAME = "QLMuaBanXe";
    private static final String DEFAULT_SQLCMD_SERVER = ".\\SQL2025DEV";

    private final String jdbcUrl;
    private final String username;
    private final String password;
    private final List<String> candidateSqlcmdServers;
    private String activeSqlcmdServer;

    private boolean forceDisconnected = false;
    private boolean simulatedDatabaseMode = false;
    private Boolean useSqlcmdBackend = null;

    // Bộ nhớ dữ liệu ảo / giả lập khi mất kết nối hoặc khi chạy trên máy không cài SQL Server
    private final List<KhachHang> virtualKhachHang = new ArrayList<KhachHang>();
    private final List<Xe> virtualXe = new ArrayList<Xe>();
    private final List<ChiTietMuaXe> virtualMuaXe = new ArrayList<ChiTietMuaXe>();

    public MuaBanXeRepository() {
        Properties props = taiCauHinhTuFileVaMoiTruong();
        String host = layGiaTriCauHinh(props, "db.host", "DB_HOST", DEFAULT_HOST);
        String port = layGiaTriCauHinh(props, "db.port", "DB_PORT", DEFAULT_PORT);
        String dbName = layGiaTriCauHinh(props, "db.name", "DB_NAME", DEFAULT_DB_NAME);
        String integrated = layGiaTriCauHinh(props, "db.integratedSecurity", "DB_INTEGRATED_SECURITY", "true");
        String customUrl = layGiaTriCauHinh(props, "db.url", "DB_URL", "");

        if (!customUrl.isEmpty()) {
            this.jdbcUrl = customUrl;
        } else {
            boolean useIntegrated = "true".equalsIgnoreCase(integrated) || "1".equals(integrated);
            StringBuilder urlBuilder = new StringBuilder();
            urlBuilder.append("jdbc:sqlserver://").append(host).append(":").append(port)
                    .append(";databaseName=").append(dbName)
                    .append(";encrypt=true;trustServerCertificate=true;loginTimeout=2");
            if (useIntegrated) {
```

```
                urlBuilder.append(";integratedSecurity=true");
            }
            this.jdbcUrl = urlBuilder.toString();
        }

        this.username = layGiaTriCauHinh(props, "db.user", "DB_USER", "");
        this.password = layGiaTriCauHinh(props, "db.password", "DB_PASSWORD", "");

        String serversRaw = layGiaTriCauHinh(
                props,
                "db.sqlcmdServers",
                "DB_SQLCMD_SERVER",
                DEFAULT_SQLCMD_SERVER + ",localhost,.,.\\SQLEXPRESS,(local)"
        );
        this.candidateSqlcmdServers = phanTachDanhSachServer(serversRaw);
        this.activeSqlcmdServer = this.candidateSqlcmdServers.isEmpty()
                ? DEFAULT_SQLCMD_SERVER
                : this.candidateSqlcmdServers.get(0);
    }

    public MuaBanXeRepository(String jdbcUrl, String username, String password, String sqlcmdServer) {
        this.jdbcUrl = jdbcUrl == null || jdbcUrl.trim().isEmpty()
                ?
"jdbc:sqlserver://127.0.0.1:1433;databaseName=QLMuaBanXe;encrypt=true;trustServerCertificate=true;integratedSecu
rity=true;loginTimeout=2"
                : jdbcUrl.trim();
        this.username = username == null ? "" : username.trim();
        this.password = password == null ? "" : password;
        String serverCfg = sqlcmdServer == null || sqlcmdServer.trim().isEmpty()
                ? DEFAULT_SQLCMD_SERVER + ",localhost,.,.\\SQLEXPRESS,(local)"
                : sqlcmdServer.trim();
        this.candidateSqlcmdServers = phanTachDanhSachServer(serverCfg);
        this.activeSqlcmdServer = this.candidateSqlcmdServers.isEmpty()
                ? DEFAULT_SQLCMD_SERVER
                : this.candidateSqlcmdServers.get(0);
    }

    private static Properties taiCauHinhTuFileVaMoiTruong() {
        Properties props = new Properties();
        String[] candidateFiles = new String[]{
                "config.properties",
                "db.properties",
                "Kiểm tra cuối kì/config.properties",
                "../config.properties"
        };
        for (int i = 0; i < candidateFiles.length; i++) {
            File f = new File(candidateFiles[i]);
            if (f.isFile()) {
                FileInputStream fis = null;
                try {
                    fis = new FileInputStream(f);
                    props.load(fis);
                    break;
                } catch (Exception ignored) {
                } finally {
                    if (fis != null) {
                        try {
                            fis.close();
                        } catch (Exception ignored) {
                        }
                    }
                }
```

```
            }
        }
        return props;
    }

    private static String layGiaTriCauHinh(Properties props, String propKey, String envKey, String defaultVal) {
        String sysVal = System.getProperty(propKey);
        if (sysVal != null && !sysVal.trim().isEmpty()) {
            return sysVal.trim();
        }
        String envVal = System.getenv(envKey);
        if (envVal != null && !envVal.trim().isEmpty()) {
            return envVal.trim();
        }
        String fileVal = props.getProperty(propKey);
        if (fileVal != null && !fileVal.trim().isEmpty()) {
            return fileVal.trim();
        }
        return defaultVal;
    }

    private static List<String> phanTachDanhSachServer(String raw) {
        List<String> list = new ArrayList<String>();
        if (raw != null) {
            String[] parts = raw.split(",");
            for (int i = 0; i < parts.length; i++) {
                String s = parts[i].trim();
                if (!s.isEmpty() && !list.contains(s)) {
                    list.add(s);
                }
            }
        }
        if (list.isEmpty()) {
            list.add(DEFAULT_SQLCMD_SERVER);
        }
        return list;
    }

    public void setForceDisconnected(boolean forceDisconnected) {
        this.forceDisconnected = forceDisconnected;
        if (!this.simulatedDatabaseMode) {
            this.useSqlcmdBackend = null;
        }
    }

    public boolean isForceDisconnected() {
        return forceDisconnected;
    }

    /**
     * Kích hoạt chế độ giả lập CSDL (Simulated Database Mode) với đầy đủ 4 Xe và 6 giao dịch mẫu
     * từ QLMuaBanXe.sql khi chạy kiểm thử trên máy chấm bài không cài đặt SQL Server.
     */
    public void enableSimulatedDatabaseMode() {
        this.simulatedDatabaseMode = true;
        this.forceDisconnected = false;
        khoiTaoDuLieuMauAoDayDu();
    }

    public void disableSimulatedDatabaseMode() {
        this.simulatedDatabaseMode = false;
        this.useSqlcmdBackend = null;
```

```
    }

    public boolean isSimulatedDatabaseMode() {
        return simulatedDatabaseMode;
    }

    /**
     * Kiểm tra kết nối tới CSDL SQL Server QLMuaBanXe có thành công hay không.
     */
    public boolean kiemTraKetNoi() {
        if (forceDisconnected) {
            return false;
        }
        if (simulatedDatabaseMode) {
            return true;
        }
        try {
            if (useSqlcmdBackend != null) {
                if (!useSqlcmdBackend.booleanValue()) {
                    Connection conn = null;
                    try {
                        conn = openJdbc();
                        return conn != null && !conn.isClosed();
                    } finally {
                        closeQuietly(conn);
                    }
                } else {
                    return "1".equals(sqlcmdSingleValue("SELECT 1"));
                }
            }
            Connection conn = null;
            try {
                conn = openJdbc();
                if (conn != null && !conn.isClosed()) {
                    useSqlcmdBackend = Boolean.FALSE;
                    return true;
                }
            } catch (Throwable ignored) {
                // Chuyển sang dò tìm các instance SQL Server qua sqlcmd cục bộ
            } finally {
                closeQuietly(conn);
            }
            for (int i = 0; i < candidateSqlcmdServers.size(); i++) {
                String candidate = candidateSqlcmdServers.get(i);
                this.activeSqlcmdServer = candidate;
                try {
                    if ("1".equals(sqlcmdSingleValue("SELECT 1"))) {
                        useSqlcmdBackend = Boolean.TRUE;
                        return true;
                    }
                } catch (Throwable ignored) {
                }
            }
        } catch (Throwable ignored) {
            // Kết nối thất bại
        }
        return false;
    }

    /**
     * Khởi tạo dòng dữ liệu ảo dự phòng (Virtual Row) khi không kết nối được Database Server
     * theo đúng yêu cầu tại phần Ghi chú trang 3 của đề thi.
```

```
     */
    public void khoiTaoDongDuLieuAo() {
        khoiTaoDongDuLieuAo(null);
    }

    /**
     * Khởi tạo dòng dữ liệu ảo dự phòng khi mất kết nối Database Server, đồng thời giữ lại
     * thông tin Xe người dùng vừa nhập trên biểu mẫu (nếu hợp lệ).
     */
    public void khoiTaoDongDuLieuAo(Xe customXe) {
        virtualKhachHang.clear();
        virtualXe.clear();
        virtualMuaXe.clear();

        KhachHang khAo = new KhachHang("KH01", "Nguyễn Văn Sơn (Dữ liệu ảo)", "301458080", "Nam", "125 Nguyễn
Trãi, Q.1, TP.HCM");
        KhachHang khAo2 = new KhachHang("KH02", "Trần Thị Mai (Dữ liệu ảo)", "079199001", "Nữ", "42 Lê Lợi, Q.3,
TP.HCM");
        virtualKhachHang.add(khAo);
        virtualKhachHang.add(khAo2);

        Xe xeAo = new Xe("XE01", "Suzuki X-Bike", 125, 21500000L);
        virtualXe.add(xeAo);

        ChiTietMuaXe ctAo = new ChiTietMuaXe(
                khAo.getMaKH(), khAo.getTenKH(), khAo.getCmnd(), khAo.getGioiTinh(), khAo.getDiaChi(),
                xeAo.getMaXe(), "28/10/2011", 1, xeAo.getDonGia(), "Dòng dữ liệu ảo khi mất kết nối CSDL"
        );
        virtualMuaXe.add(ctAo);

        if (customXe != null && !customXe.getMaXe().isEmpty() && !customXe.getMaXe().equalsIgnoreCase("XE01")) {
            virtualXe.add(customXe);
        }
    }

    /**
     * Khởi tạo đầy đủ 4 xe mẫu và 6 giao dịch mua xe như trong QLMuaBanXe.sql
     * phục vụ chạy kiểm thử trên môi trường OnlineGDB hoặc máy không có SQL Server.
     */
    public void khoiTaoDuLieuMauAoDayDu() {
        virtualKhachHang.clear();
        virtualXe.clear();
        virtualMuaXe.clear();

        virtualKhachHang.add(new KhachHang("KH01", "Nguyễn Văn Sơn", "301458080", "Nam", "125 Nguyễn Trãi, Q.1,
TP.HCM"));
        virtualKhachHang.add(new KhachHang("KH02", "Trần Thị Mai", "079199001", "Nữ", "42 Lê Lợi, Q.3,
TP.HCM"));
        virtualKhachHang.add(new KhachHang("KH03", "Lê Hoàng Nam", "079198234", "Nam", "88 Cộng Hòa, Q.Tân Bình,
TP.HCM"));
        virtualKhachHang.add(new KhachHang("KH04", "Phạm Minh Tuấn", "079200567", "Nam", "19 Võ Văn Ngân, TP.Thủ
Đức"));

        virtualXe.add(new Xe("XE01", "Suzuki X-Bike", 125, 21500000L));
        virtualXe.add(new Xe("XE02", "Honda AirBlade", 150, 42000000L));
        virtualXe.add(new Xe("XE03", "Yamaha Exciter", 155, 47500000L));
        virtualXe.add(new Xe("KH001", "Towner", 110, 18500000L));

        virtualMuaXe.add(new ChiTietMuaXe("KH01", "Nguyễn Văn Sơn", "301458080", "Nam", "125 Nguyễn Trãi, Q.1,
TP.HCM",
                "XE01", "28/10/2011", 1, 21500000L, "Khách mua tiền mặt"));
        virtualMuaXe.add(new ChiTietMuaXe("KH02", "Trần Thị Mai", "079199001", "Nữ", "42 Lê Lợi, Q.3, TP.HCM",
```

```
                "XE02", "15/05/2012", 1, 42000000L, "Tặng mũ bảo hiểm"));
        virtualMuaXe.add(new ChiTietMuaXe("KH03", "Lê Hoàng Nam", "079198234", "Nam", "88 Cộng Hòa, Q.Tân Bình,
TP.HCM",
                "XE02", "20/06/2012", 2, 42000000L, "Mua theo lô công ty"));
        virtualMuaXe.add(new ChiTietMuaXe("KH04", "Phạm Minh Tuấn", "079200567", "Nam", "19 Võ Văn Ngân, TP.Thủ
Đức",
                "XE03", "12/08/2012", 1, 47500000L, "Bảo hành 3 năm"));
        virtualMuaXe.add(new ChiTietMuaXe("KH01", "Nguyễn Văn Sơn", "301458080", "Nam", "125 Nguyễn Trãi, Q.1,
TP.HCM",
                "KH001", "29/10/2012", 3, 18500000L, ""));
        virtualMuaXe.add(new ChiTietMuaXe("KH02", "Trần Thị Mai", "079199001", "Nữ", "42 Lê Lợi, Q.3, TP.HCM",
                "KH001", "28/10/2012", 1, 18500000L, "Laptop"));
    }

    private List<Xe> sapXepDanhSachXe(List<Xe> input) {
        List<Xe> copy = new ArrayList<Xe>(input);
        Collections.sort(copy, new Comparator<Xe>() {
            @Override
            public int compare(Xe a, Xe b) {
                boolean aXe = a.getMaXe().toUpperCase().startsWith("XE");
                boolean bXe = b.getMaXe().toUpperCase().startsWith("XE");
                if (aXe != bXe) {
                    return aXe ? -1 : 1;
                }
                return a.getMaXe().compareToIgnoreCase(b.getMaXe());
            }
        });
        return copy;
    }

    private List<ChiTietMuaXe> sapXepChiTietMua(List<ChiTietMuaXe> input) {
        List<ChiTietMuaXe> copy = new ArrayList<ChiTietMuaXe>(input);
        Collections.sort(copy, new Comparator<ChiTietMuaXe>() {
            @Override
            public int compare(ChiTietMuaXe a, ChiTietMuaXe b) {
                try {
                    String isoA = chuyenSangIsoDate(a.getNgayMua());
                    String isoB = chuyenSangIsoDate(b.getNgayMua());
                    int cmpDate = isoB.compareTo(isoA); // DESC
                    if (cmpDate != 0) {
                        return cmpDate;
                    }
                } catch (SQLException ignored) {
                }
                return a.getMaKH().compareToIgnoreCase(b.getMaKH());
            }
        });
        return copy;
    }

    public List<Xe> layDanhSachXe() throws SQLException {
        if (!kiemTraKetNoi() || simulatedDatabaseMode) {
            if (virtualXe.isEmpty()) {
                khoiTaoDongDuLieuAo();
            }
            return sapXepDanhSachXe(virtualXe);
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "SELECT MaXe, " + hexCol("TenXe") + ", DungTich, DonGia FROM dbo.Xe "
                    + "ORDER BY CASE WHEN MaXe LIKE 'XE%' THEN 0 ELSE 1 END, MaXe";
            List<String[]> rows = sqlcmdQuery(sql, 4);
            List<Xe> list = new ArrayList<Xe>();
```

```
            for (int i = 0; i < rows.size(); i++) {
                String[] r = rows.get(i);
                list.add(new Xe(r[0], decodeHexUnicode(r[1]), Integer.parseInt(r[2]), Long.parseLong(r[3])));
            }
            return list;
        }
        String sql = "SELECT MaXe, TenXe, DungTich, DonGia FROM dbo.Xe "
                + "ORDER BY CASE WHEN MaXe LIKE 'XE%' THEN 0 ELSE 1 END, MaXe";
        List<Xe> list = new ArrayList<Xe>();
        Connection c = null;
        PreparedStatement ps = null;
        ResultSet rs = null;
        try {
            c = openJdbc();
            ps = c.prepareStatement(sql);
            rs = ps.executeQuery();
            while (rs.next()) {
                list.add(new Xe(rs.getString(1), rs.getString(2), rs.getInt(3), rs.getLong(4)));
            }
        } finally {
            closeQuietly(rs);
            closeQuietly(ps);
            closeQuietly(c);
        }
        return list;
    }

    public List<KhachHang> layDanhSachKhachHang() throws SQLException {
        if (!kiemTraKetNoi() || simulatedDatabaseMode) {
            if (virtualKhachHang.isEmpty()) {
                khoiTaoDongDuLieuAo();
            }
            return new ArrayList<KhachHang>(virtualKhachHang);
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "SELECT MaKH, " + hexCol("TenKH") + ", CMND, " + hexCol("GioiTinh") + ", "
                    + hexCol("DiaChi") + " FROM dbo.Khach_Hang ORDER BY MaKH";
            List<String[]> rows = sqlcmdQuery(sql, 5);
            List<KhachHang> list = new ArrayList<KhachHang>();
            for (int i = 0; i < rows.size(); i++) {
                String[] r = rows.get(i);
                list.add(new KhachHang(r[0], decodeHexUnicode(r[1]), r[2],
                        decodeHexUnicode(r[3]), decodeHexUnicode(r[4])));
            }
            return list;
        }
        String sql = "SELECT MaKH, TenKH, CMND, GioiTinh, DiaChi FROM dbo.Khach_Hang ORDER BY MaKH";
        List<KhachHang> list = new ArrayList<KhachHang>();
        Connection c = null;
        PreparedStatement ps = null;
        ResultSet rs = null;
        try {
            c = openJdbc();
            ps = c.prepareStatement(sql);
            rs = ps.executeQuery();
            while (rs.next()) {
                list.add(new KhachHang(rs.getString(1), rs.getString(2), rs.getString(3),
                        rs.getString(4), rs.getString(5)));
            }
        } finally {
            closeQuietly(rs);
            closeQuietly(ps);
```

```
            closeQuietly(c);
        }
        return list;
    }

    public List<ChiTietMuaXe> layChiTietMuaTheoXe(String maXe) throws SQLException {
        if (!kiemTraKetNoi() || simulatedDatabaseMode) {
            List<ChiTietMuaXe> list = new ArrayList<ChiTietMuaXe>();
            for (int i = 0; i < virtualMuaXe.size(); i++) {
                ChiTietMuaXe ct = virtualMuaXe.get(i);
                if (ct.getMaXe().equalsIgnoreCase(maXe)) {
                    list.add(ct);
                }
            }
            return sapXepChiTietMua(list);
        }
        String cleanMaXe = sanitizeSqlLiteral(maXe);
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "SELECT M.MaKH, " + hexCol("K.TenKH") + ", K.CMND, " + hexCol("K.GioiTinh") + ", "
                    + hexCol("K.DiaChi") + ", M.MaXe, CONVERT(varchar(10), M.NgayMua, 103), M.SoLuong, "
                    + "X.DonGia, " + hexCol("M.GhiChu") + " "
                    + "FROM dbo.Mua_Xe M "
                    + "JOIN dbo.Khach_Hang K ON M.MaKH = K.MaKH "
                    + "JOIN dbo.Xe X ON M.MaXe = X.MaXe "
                    + "WHERE M.MaXe = '" + cleanMaXe + "' "
                    + "ORDER BY M.NgayMua DESC, M.MaKH ASC";
            List<String[]> rows = sqlcmdQuery(sql, 10);
            List<ChiTietMuaXe> list = new ArrayList<ChiTietMuaXe>();
            for (int i = 0; i < rows.size(); i++) {
                String[] r = rows.get(i);
                list.add(new ChiTietMuaXe(
                        r[0], decodeHexUnicode(r[1]), r[2], decodeHexUnicode(r[3]), decodeHexUnicode(r[4]),
                        r[5], r[6], Integer.parseInt(r[7]), Long.parseLong(r[8]), decodeHexUnicode(r[9])
                ));
            }
            return list;
        }
        String sql = "SELECT M.MaKH, K.TenKH, K.CMND, K.GioiTinh, K.DiaChi, M.MaXe, "
                + "CONVERT(varchar(10), M.NgayMua, 103), M.SoLuong, X.DonGia, M.GhiChu "
                + "FROM dbo.Mua_Xe M "
                + "JOIN dbo.Khach_Hang K ON M.MaKH = K.MaKH "
                + "JOIN dbo.Xe X ON M.MaXe = X.MaXe "
                + "WHERE M.MaXe = ? ORDER BY M.NgayMua DESC, M.MaKH ASC";
        List<ChiTietMuaXe> list = new ArrayList<ChiTietMuaXe>();
        Connection c = null;
        PreparedStatement ps = null;
        ResultSet rs = null;
        try {
            c = openJdbc();
            ps = c.prepareStatement(sql);
            ps.setString(1, maXe);
            rs = ps.executeQuery();
            while (rs.next()) {
                list.add(new ChiTietMuaXe(
                        rs.getString(1), rs.getString(2), rs.getString(3), rs.getString(4), rs.getString(5),
                        rs.getString(6), rs.getString(7), rs.getInt(8), rs.getLong(9), rs.getString(10)
                ));
            }
        } finally {
            closeQuietly(rs);
            closeQuietly(ps);
            closeQuietly(c);
```

```
        }
        return list;
    }

    /**
     * Lọc dữ liệu báo cáo theo Mã xe và Ngày mua (dd/MM/yyyy).
     * Nếu người dùng nhập Ngày mua thì bắt buộc kiểm tra đúng định dạng dd/MM/yyyy và so khớp chính xác ngày.
     */
    public List<ChiTietMuaXe> locBaoCaoMuaXe(String maXe, String ngayMuaFilter) throws SQLException {
        String filter = ngayMuaFilter == null ? "" : ngayMuaFilter.trim();
        String normalizedFilterDate = "";
        if (!filter.isEmpty()) {
            normalizedFilterDate = chuanHoaNgayDdMmYyyy(filter);
        }

        List<ChiTietMuaXe> source = new ArrayList<ChiTietMuaXe>();
        if (maXe == null || maXe.trim().isEmpty() || "-- Tất cả --".equals(maXe.trim())) {
            List<Xe> allXe = layDanhSachXe();
            for (int i = 0; i < allXe.size(); i++) {
                source.addAll(layChiTietMuaTheoXe(allXe.get(i).getMaXe()));
            }
        } else {
            source.addAll(layChiTietMuaTheoXe(maXe.trim()));
        }

        if (normalizedFilterDate.isEmpty()) {
            return source;
        }
        List<ChiTietMuaXe> filtered = new ArrayList<ChiTietMuaXe>();
        for (int i = 0; i < source.size(); i++) {
            ChiTietMuaXe ct = source.get(i);
            String ctDate = ct.getNgayMua();
            try {
                ctDate = chuanHoaNgayDdMmYyyy(ctDate);
            } catch (SQLException ignored) {
            }
            if (ctDate.equalsIgnoreCase(normalizedFilterDate)) {
                filtered.add(ct);
            }
        }
        return filtered;
    }

    public boolean tonTaiMaXe(String maXe) throws SQLException {
        List<Xe> all = layDanhSachXe();
        for (int i = 0; i < all.size(); i++) {
            if (all.get(i).getMaXe().equalsIgnoreCase(maXe.trim())) {
                return true;
            }
        }
        return false;
    }

    public void themXe(Xe xe) throws SQLException {
        if (tonTaiMaXe(xe.getMaXe())) {
            throw new SQLException("Mã xe đã tồn tại, vui lòng nhập lại!");
        }
        if (!kiemTraKetNoi() || simulatedDatabaseMode) {
            virtualXe.add(xe);
            return;
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
```

```
            String sql = "INSERT INTO dbo.Xe (MaXe, TenXe, DungTich, DonGia) VALUES ('"
                    + sanitizeSqlLiteral(xe.getMaXe()) + "', "
                    + toNCharExpr(xe.getTenXe()) + ", "
                    + xe.getDungTich() + ", "
                    + xe.getDonGia() + "); SELECT 'OK'";
            if (!"OK".equals(sqlcmdSingleValue(sql))) {
                throw new SQLException("Không thể thêm Xe vào CSDL.");
            }
            return;
        }
        String sql = "INSERT INTO dbo.Xe (MaXe, TenXe, DungTich, DonGia) VALUES (?, ?, ?, ?)";
        Connection c = null;
        PreparedStatement ps = null;
        try {
            c = openJdbc();
            ps = c.prepareStatement(sql);
            ps.setString(1, xe.getMaXe());
            ps.setString(2, xe.getTenXe());
            ps.setInt(3, xe.getDungTich());
            ps.setLong(4, xe.getDonGia());
            ps.executeUpdate();
        } finally {
            closeQuietly(ps);
            closeQuietly(c);
        }
    }

    public void xoaXe(String maXe) throws SQLException {
        if (!kiemTraKetNoi() || simulatedDatabaseMode) {
            for (int i = virtualXe.size() - 1; i >= 0; i--) {
                if (virtualXe.get(i).getMaXe().equalsIgnoreCase(maXe)) {
                    virtualXe.remove(i);
                }
            }
            for (int i = virtualMuaXe.size() - 1; i >= 0; i--) {
                if (virtualMuaXe.get(i).getMaXe().equalsIgnoreCase(maXe)) {
                    virtualMuaXe.remove(i);
                }
            }
            return;
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String clean = sanitizeSqlLiteral(maXe);
            String sql = "DELETE FROM dbo.Mua_Xe WHERE MaXe = '" + clean + "'; "
                    + "DELETE FROM dbo.Xe WHERE MaXe = '" + clean + "'; SELECT 'OK'";
            sqlcmdSingleValue(sql);
            return;
        }
        Connection c = null;
        PreparedStatement ps1 = null;
        PreparedStatement ps2 = null;
        try {
            c = openJdbc();
            ps1 = c.prepareStatement("DELETE FROM dbo.Mua_Xe WHERE MaXe = ?");
            ps1.setString(1, maXe);
            ps1.executeUpdate();

            ps2 = c.prepareStatement("DELETE FROM dbo.Xe WHERE MaXe = ?");
            ps2.setString(1, maXe);
            ps2.executeUpdate();
        } finally {
            closeQuietly(ps1);
```

```
            closeQuietly(ps2);
            closeQuietly(c);
        }
    }

    /**
     * Lưu mới hoặc cập nhật thông tin Khách hàng.
     * Nếu khách hàng đã tồn tại và địa chỉ truyền vào bị rỗng, giữ nguyên địa chỉ hiện có trong CSDL
     * thay vì ghi đè bằng giá trị mặc định.
     */
    public void luuHoacCapNhatKhachHang(KhachHang kh) throws SQLException {
        String diaChiMoi = kh.getDiaChi() == null ? "" : kh.getDiaChi().trim();
        if (!kiemTraKetNoi() || simulatedDatabaseMode) {
            for (int i = 0; i < virtualKhachHang.size(); i++) {
                KhachHang existing = virtualKhachHang.get(i);
                if (existing.getMaKH().equalsIgnoreCase(kh.getMaKH())) {
                    String resolvedAddress = diaChiMoi.isEmpty() ? existing.getDiaChi() : diaChiMoi;
                    if (resolvedAddress.isEmpty()) {
                        resolvedAddress = "TP.HCM";
                    }
                    KhachHang updatedKh = new KhachHang(
                            kh.getMaKH(), kh.getTenKH(), kh.getCmnd(), kh.getGioiTinh(), resolvedAddress
                    );
                    virtualKhachHang.set(i, updatedKh);
                    dongBoKhachHangTrongVirtualMuaXe(updatedKh);
                    return;
                }
            }
            String resolvedAddress = diaChiMoi.isEmpty() ? "TP.HCM" : diaChiMoi;
            KhachHang newKh = new KhachHang(
                    kh.getMaKH(), kh.getTenKH(), kh.getCmnd(), kh.getGioiTinh(), resolvedAddress
            );
            virtualKhachHang.add(newKh);
            dongBoKhachHangTrongVirtualMuaXe(newKh);
            return;
        }

        String cleanMa = sanitizeSqlLiteral(kh.getMaKH());
        String cleanCmnd = sanitizeSqlLiteral(kh.getCmnd());
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String updateDiaChiClause = diaChiMoi.isEmpty()
                    ? ""
                    : ", DiaChi = " + toNCharExpr(diaChiMoi);
            String insertDiaChiExpr = toNCharExpr(diaChiMoi.isEmpty() ? "TP.HCM" : diaChiMoi);
            String sql = "IF EXISTS (SELECT 1 FROM dbo.Khach_Hang WHERE MaKH = '" + cleanMa + "') "
                    + "UPDATE dbo.Khach_Hang SET TenKH = " + toNCharExpr(kh.getTenKH())
                    + ", CMND = '" + cleanCmnd + "', GioiTinh = " + toNCharExpr(kh.getGioiTinh())
                    + updateDiaChiClause
                    + " WHERE MaKH = '" + cleanMa + "' "
                    + "ELSE INSERT INTO dbo.Khach_Hang (MaKH, TenKH, CMND, GioiTinh, DiaChi) VALUES ('"
                    + cleanMa + "', " + toNCharExpr(kh.getTenKH()) + ", '" + cleanCmnd + "', "
                    + toNCharExpr(kh.getGioiTinh()) + ", "
                    + insertDiaChiExpr + "); SELECT 'OK'";
            sqlcmdSingleValue(sql);
            return;
        }

        Connection c = null;
        PreparedStatement checkPs = null;
        ResultSet rs = null;
        PreparedStatement actionPs = null;
        try {
```

```
            c = openJdbc();
            checkPs = c.prepareStatement("SELECT DiaChi FROM dbo.Khach_Hang WHERE MaKH = ?");
            checkPs.setString(1, kh.getMaKH());
            rs = checkPs.executeQuery();
            boolean exists = false;
            String existingDiaChi = "";
            if (rs.next()) {
                exists = true;
                existingDiaChi = rs.getString(1) == null ? "" : rs.getString(1).trim();
            }
            if (exists) {
                String finalDiaChi = diaChiMoi.isEmpty()
                        ? (existingDiaChi.isEmpty() ? "TP.HCM" : existingDiaChi)
                        : diaChiMoi;
                actionPs = c.prepareStatement(
                        "UPDATE dbo.Khach_Hang SET TenKH=?, CMND=?, GioiTinh=?, DiaChi=? WHERE MaKH=?"
                );
                actionPs.setString(1, kh.getTenKH());
                actionPs.setString(2, kh.getCmnd());
                actionPs.setString(3, kh.getGioiTinh());
                actionPs.setString(4, finalDiaChi);
                actionPs.setString(5, kh.getMaKH());
                actionPs.executeUpdate();
            } else {
                String finalDiaChi = diaChiMoi.isEmpty() ? "TP.HCM" : diaChiMoi;
                actionPs = c.prepareStatement(
                        "INSERT INTO dbo.Khach_Hang (MaKH, TenKH, CMND, GioiTinh, DiaChi) VALUES (?, ?, ?, ?,
?)"
                );
                actionPs.setString(1, kh.getMaKH());
                actionPs.setString(2, kh.getTenKH());
                actionPs.setString(3, kh.getCmnd());
                actionPs.setString(4, kh.getGioiTinh());
                actionPs.setString(5, finalDiaChi);
                actionPs.executeUpdate();
            }
        } finally {
            closeQuietly(rs);
            closeQuietly(checkPs);
            closeQuietly(actionPs);
            closeQuietly(c);
        }
    }

    private void dongBoKhachHangTrongVirtualMuaXe(KhachHang kh) {
        for (int i = 0; i < virtualMuaXe.size(); i++) {
            ChiTietMuaXe ct = virtualMuaXe.get(i);
            if (ct.getMaKH().equalsIgnoreCase(kh.getMaKH())) {
                ct.setTenKH(kh.getTenKH());
                ct.setCmnd(kh.getCmnd());
                ct.setGioiTinh(kh.getGioiTinh());
                ct.setDiaChi(kh.getDiaChi());
            }
        }
    }

    public void themChiTietMuaXe(ChiTietMuaXe ct) throws SQLException {
        String isoDate = chuyenSangIsoDate(ct.getNgayMua());
        String normalizedNgay = chuanHoaNgayDdMmYyyy(ct.getNgayMua());
        ct.setNgayMua(normalizedNgay);

        luuHoacCapNhatKhachHang(new KhachHang(
```

```
                ct.getMaKH(), ct.getTenKH(), ct.getCmnd(), ct.getGioiTinh(), ct.getDiaChi()
        ));
        if (!kiemTraKetNoi() || simulatedDatabaseMode) {
            for (int i = 0; i < virtualMuaXe.size(); i++) {
                ChiTietMuaXe item = virtualMuaXe.get(i);
                if (item.getMaKH().equalsIgnoreCase(ct.getMaKH())
                        && item.getMaXe().equalsIgnoreCase(ct.getMaXe())
                        && item.getNgayMua().equalsIgnoreCase(ct.getNgayMua())) {
                    throw new SQLException("Khách hàng '" + ct.getMaKH() + "' đã có giao dịch mua xe '"
                            + ct.getMaXe() + "' vào ngày " + ct.getNgayMua() + "!");
                }
            }
            // Đồng bộ lại địa chỉ từ danh sách khách hàng nếu ct.diaChi đang để trống
            if (ct.getDiaChi().isEmpty()) {
                for (int i = 0; i < virtualKhachHang.size(); i++) {
                    if (virtualKhachHang.get(i).getMaKH().equalsIgnoreCase(ct.getMaKH())) {
                        ct.setDiaChi(virtualKhachHang.get(i).getDiaChi());
                        break;
                    }
                }
            }
            virtualMuaXe.add(ct);
            return;
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "INSERT INTO dbo.Mua_Xe (MaKH, MaXe, NgayMua, SoLuong, GhiChu) VALUES ('"
                    + sanitizeSqlLiteral(ct.getMaKH()) + "', '"
                    + sanitizeSqlLiteral(ct.getMaXe()) + "', '"
                    + isoDate + "', "
                    + ct.getSoLuong() + ", "
                    + toNCharExpr(ct.getGhiChu()) + "); SELECT 'OK'";
            sqlcmdSingleValue(sql);
            return;
        }
        String sql = "INSERT INTO dbo.Mua_Xe (MaKH, MaXe, NgayMua, SoLuong, GhiChu) VALUES (?, ?, ?, ?, ?)";
        Connection c = null;
        PreparedStatement ps = null;
        try {
            c = openJdbc();
            ps = c.prepareStatement(sql);
            ps.setString(1, ct.getMaKH());
            ps.setString(2, ct.getMaXe());
            ps.setString(3, isoDate);
            ps.setInt(4, ct.getSoLuong());
            ps.setString(5, ct.getGhiChu());
            ps.executeUpdate();
        } finally {
            closeQuietly(ps);
            closeQuietly(c);
        }
    }

    public void suaChiTietMuaXe(String oldMaKH, String maXe, String oldNgayMua, ChiTietMuaXe updated) throws
SQLException {
        String oldIso = chuyenSangIsoDate(oldNgayMua);
        String newIso = chuyenSangIsoDate(updated.getNgayMua());
        String normalizedNewNgay = chuanHoaNgayDdMmYyyy(updated.getNgayMua());
        updated.setNgayMua(normalizedNewNgay);

        luuHoacCapNhatKhachHang(new KhachHang(
                updated.getMaKH(), updated.getTenKH(), updated.getCmnd(), updated.getGioiTinh(),
updated.getDiaChi()
```

```
        ));
        if (!kiemTraKetNoi() || simulatedDatabaseMode) {
            for (int i = 0; i < virtualMuaXe.size(); i++) {
                ChiTietMuaXe item = virtualMuaXe.get(i);
                if (item.getMaKH().equalsIgnoreCase(oldMaKH)
                        && item.getMaXe().equalsIgnoreCase(maXe)
                        && item.getNgayMua().equalsIgnoreCase(oldNgayMua)) {
                    if (updated.getDiaChi().isEmpty()) {
                        updated.setDiaChi(item.getDiaChi());
                    }
                    virtualMuaXe.set(i, updated);
                    return;
                }
            }
            return;
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "UPDATE dbo.Mua_Xe SET MaKH = '" + sanitizeSqlLiteral(updated.getMaKH())
                    + "', NgayMua = '" + newIso
                    + "', SoLuong = " + updated.getSoLuong()
                    + ", GhiChu = " + toNCharExpr(updated.getGhiChu())
                    + " WHERE MaKH = '" + sanitizeSqlLiteral(oldMaKH)
                    + "' AND MaXe = '" + sanitizeSqlLiteral(maXe)
                    + "' AND NgayMua = '" + oldIso + "'; SELECT 'OK'";
            sqlcmdSingleValue(sql);
            return;
        }
        String sql = "UPDATE dbo.Mua_Xe SET MaKH=?, NgayMua=?, SoLuong=?, GhiChu=? WHERE MaKH=? AND MaXe=? AND
NgayMua=?";
        Connection c = null;
        PreparedStatement ps = null;
        try {
            c = openJdbc();
            ps = c.prepareStatement(sql);
            ps.setString(1, updated.getMaKH());
            ps.setString(2, newIso);
            ps.setInt(3, updated.getSoLuong());
            ps.setString(4, updated.getGhiChu());
            ps.setString(5, oldMaKH);
            ps.setString(6, maXe);
            ps.setString(7, oldIso);
            ps.executeUpdate();
        } finally {
            closeQuietly(ps);
            closeQuietly(c);
        }
    }

    public void xoaChiTietMuaXe(String maKH, String maXe, String ngayMua) throws SQLException {
        String isoDate = chuyenSangIsoDate(ngayMua);
        if (!kiemTraKetNoi() || simulatedDatabaseMode) {
            for (int i = virtualMuaXe.size() - 1; i >= 0; i--) {
                ChiTietMuaXe item = virtualMuaXe.get(i);
                if (item.getMaKH().equalsIgnoreCase(maKH)
                        && item.getMaXe().equalsIgnoreCase(maXe)
                        && item.getNgayMua().equalsIgnoreCase(ngayMua)) {
                    virtualMuaXe.remove(i);
                    return;
                }
            }
            return;
        }
```

```
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "DELETE FROM dbo.Mua_Xe WHERE MaKH = '" + sanitizeSqlLiteral(maKH)
                    + "' AND MaXe = '" + sanitizeSqlLiteral(maXe)
                    + "' AND NgayMua = '" + isoDate + "'; SELECT 'OK'";
            sqlcmdSingleValue(sql);
            return;
        }
        String sql = "DELETE FROM dbo.Mua_Xe WHERE MaKH=? AND MaXe=? AND NgayMua=?";
        Connection c = null;
        PreparedStatement ps = null;
        try {
            c = openJdbc();
            ps = c.prepareStatement(sql);
            ps.setString(1, maKH);
            ps.setString(2, maXe);
            ps.setString(3, isoDate);
            ps.executeUpdate();
        } finally {
            closeQuietly(ps);
            closeQuietly(c);
        }
    }

    public static String chuyenSangIsoDate(String ddMMyyyy) throws SQLException {
        if (ddMMyyyy == null || ddMMyyyy.trim().isEmpty()) {
            throw new SQLException("Ngày mua không được để trống!");
        }
        try {
            SimpleDateFormat inFmt = new SimpleDateFormat("dd/MM/yyyy");
            inFmt.setLenient(false);
            Date parsed = inFmt.parse(ddMMyyyy.trim());
            return new SimpleDateFormat("yyyy-MM-dd").format(parsed);
        } catch (ParseException e) {
            throw new SQLException("Ngày mua không đúng định dạng dd/MM/yyyy: " + ddMMyyyy);
        }
    }

    public static String chuanHoaNgayDdMmYyyy(String ddMMyyyy) throws SQLException {
        if (ddMMyyyy == null || ddMMyyyy.trim().isEmpty()) {
            throw new SQLException("Ngày mua không được để trống!");
        }
        try {
            SimpleDateFormat inFmt = new SimpleDateFormat("dd/MM/yyyy");
            inFmt.setLenient(false);
            Date parsed = inFmt.parse(ddMMyyyy.trim());
            return new SimpleDateFormat("dd/MM/yyyy").format(parsed);
        } catch (ParseException e) {
            throw new SQLException("Ngày mua không đúng định dạng dd/MM/yyyy: " + ddMMyyyy);
        }
    }

    private Connection openJdbc() throws SQLException {
        if (username.isEmpty()) {
            return DriverManager.getConnection(jdbcUrl);
        }
        return DriverManager.getConnection(jdbcUrl, username, password);
    }

    private static void closeQuietly(AutoCloseable resource) {
        if (resource != null) {
            try {
                resource.close();
```

```
            } catch (Exception ignored) {
            }
        }
    }

    private static String hexCol(String colName) {
        return "CONVERT(varchar(400), CONVERT(varbinary(200), " + colName + "), 2)";
    }

    private static String sanitizeSqlLiteral(String val) {
        return val == null ? "" : val.trim().replace("'", "''");
    }

    private static String toNCharExpr(String value) {
        if (value == null || value.isEmpty()) {
            return "N''";
        }
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < value.length(); i++) {
            if (i > 0) {
                sb.append('+');
            }
            sb.append("NCHAR(").append((int) value.charAt(i)).append(')');
        }
        return sb.toString();
    }

    private static String decodeHexUnicode(String hex) {
        if (hex == null || hex.isEmpty()) {
            return "";
        }
        byte[] bytes = new byte[hex.length() / 2];
        for (int i = 0; i < bytes.length; i++) {
            int hi = Character.digit(hex.charAt(i * 2), 16);
            int lo = Character.digit(hex.charAt(i * 2 + 1), 16);
            bytes[i] = (byte) ((hi << 4) | lo);
        }
        return new String(bytes, StandardCharsets.UTF_16LE);
    }

    private String sqlcmdSingleValue(String sql) throws SQLException {
        List<String[]> rows = sqlcmdQuery(sql, 1);
        if (rows.isEmpty()) {
            throw new SQLException("SQL Server không trả về dữ liệu.");
        }
        return rows.get(0)[0];
    }

    private List<String[]> sqlcmdQuery(String sql, int expectedCols) throws SQLException {
        String fullQuery = "SET NOCOUNT ON; SET XACT_ABORT ON; " + sql;
        List<String> cmd = new ArrayList<String>();
        cmd.add("sqlcmd");
        cmd.add("-S");
        cmd.add(activeSqlcmdServer);
        cmd.add("-C");
        if (!username.isEmpty()) {
            cmd.add("-U");
            cmd.add(username);
            cmd.add("-P");
            cmd.add(password);
        }
        cmd.add("-d");
```

```
        cmd.add("QLMuaBanXe");
        cmd.add("-W");
        cmd.add("-h");
        cmd.add("-1");
        cmd.add("-s");
        cmd.add("|");
        cmd.add("-Q");
        cmd.add(fullQuery);

        ProcessBuilder pb = new ProcessBuilder(cmd);
        pb.redirectErrorStream(true);
        try {
            Process proc = pb.start();
            ByteArrayOutputStream buffer = new ByteArrayOutputStream();
            InputStream in = null;
            try {
                in = proc.getInputStream();
                byte[] chunk = new byte[4096];
                int read;
                while ((read = in.read(chunk)) != -1) {
                    buffer.write(chunk, 0, read);
                }
            } finally {
                closeQuietly(in);
            }
            boolean finished = proc.waitFor(10, TimeUnit.SECONDS);
            if (!finished) {
                proc.destroyForcibly();
                throw new SQLException("Quá thời gian chờ kết nối SQL Server.");
            }
            String out = new String(buffer.toByteArray(), StandardCharsets.UTF_8).trim();
            if (proc.exitValue() != 0 || out.contains("Msg ")) {
                throw new SQLException("Lỗi SQL Server: " + out);
            }
            List<String[]> rows = new ArrayList<String[]>();
            if (out.isEmpty()) {
                return rows;
            }
            String[] lines = out.split("\\R");
            for (int i = 0; i < lines.length; i++) {
                String trimmed = lines[i].trim();
                if (trimmed.isEmpty()) {
                    continue;
                }
                String[] parts = trimmed.split("\\|", -1);
                if (parts.length == expectedCols) {
                    for (int j = 0; j < parts.length; j++) {
                        parts[j] = parts[j].trim();
                    }
                    rows.add(parts);
                }
            }
            return rows;
        } catch (SQLException e) {
            throw e;
        } catch (Exception e) {
            throw new SQLException("Không thể thực thi sqlcmd: " + e.getMessage(), e);
        }
    }
}

/**
```

```
 * Lớp thực thể ChiTietMuaXe kết hợp thông tin bảng Mua_Xe với Khach_Hang và Xe
 * để hiển thị lên Khung hiển thị dữ liệu chi tiết và Cửa sổ Báo cáo.
 */
class ChiTietMuaXe {
    private String maKH;
    private String tenKH;
    private String cmnd;
    private String gioiTinh;
    private String diaChi;
    private String maXe;
    private String ngayMua; // Định dạng dd/MM/yyyy
    private int soLuong;
    private long donGiaXe;
    private String ghiChu;

    public ChiTietMuaXe() {
        this.maKH = "";
        this.tenKH = "";
        this.cmnd = "";
        this.gioiTinh = "Nam";
        this.diaChi = "";
        this.maXe = "";
        this.ngayMua = "28/10/2011";
        this.soLuong = 1;
        this.donGiaXe = 0L;
        this.ghiChu = "";
    }

    public ChiTietMuaXe(String maKH, String tenKH, String cmnd, String gioiTinh, String diaChi,
                        String maXe, String ngayMua, int soLuong, long donGiaXe, String ghiChu) {
        this.maKH = maKH == null ? "" : maKH.trim();
        this.tenKH = tenKH == null ? "" : tenKH.trim();
        this.cmnd = cmnd == null ? "" : cmnd.trim();
        this.gioiTinh = gioiTinh == null ? "Nam" : gioiTinh.trim();
        this.diaChi = diaChi == null ? "" : diaChi.trim();
        this.maXe = maXe == null ? "" : maXe.trim();
        this.ngayMua = ngayMua == null ? "" : ngayMua.trim();
        this.soLuong = soLuong;
        this.donGiaXe = donGiaXe;
        this.ghiChu = ghiChu == null ? "" : ghiChu.trim();
    }

    public String getMaKH() {
        return maKH;
    }

    public void setMaKH(String maKH) {
        this.maKH = maKH == null ? "" : maKH.trim();
    }

    public String getTenKH() {
        return tenKH;
    }

    public void setTenKH(String tenKH) {
        this.tenKH = tenKH == null ? "" : tenKH.trim();
    }

    public String getCmnd() {
        return cmnd;
    }
```

```
    public void setCmnd(String cmnd) {
        this.cmnd = cmnd == null ? "" : cmnd.trim();
    }

    public String getGioiTinh() {
        return gioiTinh;
    }

    public void setGioiTinh(String gioiTinh) {
        this.gioiTinh = gioiTinh == null ? "Nam" : gioiTinh.trim();
    }

    public String getDiaChi() {
        return diaChi;
    }

    public void setDiaChi(String diaChi) {
        this.diaChi = diaChi == null ? "" : diaChi.trim();
    }

    public String getMaXe() {
        return maXe;
    }

    public void setMaXe(String maXe) {
        this.maXe = maXe == null ? "" : maXe.trim();
    }

    public String getNgayMua() {
        return ngayMua;
    }

    public void setNgayMua(String ngayMua) {
        this.ngayMua = ngayMua == null ? "" : ngayMua.trim();
    }

    public int getSoLuong() {
        return soLuong;
    }

    public void setSoLuong(int soLuong) {
        this.soLuong = soLuong;
    }

    public long getDonGiaXe() {
        return donGiaXe;
    }

    public void setDonGiaXe(long donGiaXe) {
        this.donGiaXe = donGiaXe;
    }

    public String getGhiChu() {
        return ghiChu;
    }

    public void setGhiChu(String ghiChu) {
        this.ghiChu = ghiChu == null ? "" : ghiChu.trim();
    }

    /**
     * Tính Thành tiền của dòng mua xe:
```

```
     * Thành tiền = Số lượng * Đơn giá * 1.10 (gồm 10% thuế/phí trước bạ).
     * Ví dụ trên đề: Đơn giá = 21,500,000, Số lượng = 1 => Thành tiền = 23,650,000.
     */
    public long getThanhTien() {
        return Math.round(soLuong * (double) donGiaXe * Xe.TY_LE_THANH_TIEN);
    }
}

/**
 * Lớp thực thể Xe biểu diễn dữ liệu bảng Xe trong CSDL QLMuaBanXe.
 * Cấu trúc: Xe [MaXe; TenXe (Text, 50); DungTich (số nguyên); DonGia (số nguyên)].
 */
class Xe {
    /** Hệ số thuế VAT & phí trước bạ (110% = Đơn giá + 10% thuế): 21,500,000 * 1.10 = 23,650,000 VNĐ. */
    public static final double TY_LE_THANH_TIEN = 1.10;

    private String maXe;
    private String tenXe;
    private int dungTich;
    private long donGia;

    public Xe() {
        this.maXe = "";
        this.tenXe = "";
        this.dungTich = 110;
        this.donGia = 0L;
    }

    public Xe(String maXe, String tenXe, int dungTich, long donGia) {
        this.maXe = maXe == null ? "" : maXe.trim();
        this.tenXe = tenXe == null ? "" : tenXe.trim();
        this.dungTich = dungTich;
        this.donGia = donGia;
    }

    public String getMaXe() {
        return maXe;
    }

    public void setMaXe(String maXe) {
        this.maXe = maXe == null ? "" : maXe.trim();
    }

    public String getTenXe() {
        return tenXe;
    }

    public void setTenXe(String tenXe) {
        this.tenXe = tenXe == null ? "" : tenXe.trim();
    }

    public int getDungTich() {
        return dungTich;
    }

    public void setDungTich(int dungTich) {
        this.dungTich = dungTich;
    }

    public long getDonGia() {
        return donGia;
    }
```

```
    public void setDonGia(long donGia) {
        this.donGia = donGia;
    }

    public static String dinhDangSo(double value) {
        DecimalFormatSymbols symbols = new DecimalFormatSymbols(Locale.US);
        symbols.setGroupingSeparator(',');
        DecimalFormat df = new DecimalFormat("#,##0", symbols);
        return df.format(value);
    }

    public static long parseDonGia(String text) {
        if (text == null) {
            throw new NumberFormatException("Đơn giá rỗng");
        }
        String cleaned = text.trim().replace(",", "").replace(".", "").replace("VNĐ", "").replace("VND",
"").trim();
        return Long.parseLong(cleaned);
    }

    @Override
    public String toString() {
        return maXe + " - " + tenXe;
    }
}

/**
 * Lớp thực thể KhachHang biểu diễn dữ liệu bảng Khach_Hang trong CSDL QLMuaBanXe.
 * Cấu trúc: Khach_Hang [MaKH; TenKH (Text, 50); CMND (Số); GioiTinh (Text, 10); DiaChi (Text, 50)].
 */
class KhachHang {
    private String maKH;
    private String tenKH;
    private String cmnd;
    private String gioiTinh;
    private String diaChi;

    public KhachHang() {
        this.maKH = "";
        this.tenKH = "";
        this.cmnd = "";
        this.gioiTinh = "Nam";
        this.diaChi = "";
    }

    public KhachHang(String maKH, String tenKH, String cmnd, String gioiTinh, String diaChi) {
        this.maKH = maKH == null ? "" : maKH.trim();
        this.tenKH = tenKH == null ? "" : tenKH.trim();
        this.cmnd = cmnd == null ? "" : cmnd.trim();
        this.gioiTinh = gioiTinh == null ? "Nam" : gioiTinh.trim();
        this.diaChi = diaChi == null ? "" : diaChi.trim();
    }

    public String getMaKH() {
        return maKH;
    }

    public void setMaKH(String maKH) {
        this.maKH = maKH == null ? "" : maKH.trim();
    }
```

```
    public String getTenKH() {
        return tenKH;
    }

    public void setTenKH(String tenKH) {
        this.tenKH = tenKH == null ? "" : tenKH.trim();
    }

    public String getCmnd() {
        return cmnd;
    }

    public void setCmnd(String cmnd) {
        this.cmnd = cmnd == null ? "" : cmnd.trim();
    }

    public String getGioiTinh() {
        return gioiTinh;
    }

    public void setGioiTinh(String gioiTinh) {
        this.gioiTinh = gioiTinh == null ? "Nam" : gioiTinh.trim();
    }

    public String getDiaChi() {
        return diaChi;
    }

    public void setDiaChi(String diaChi) {
        this.diaChi = diaChi == null ? "" : diaChi.trim();
    }

    @Override
    public String toString() {
        return maKH + " - " + tenKH;
    }
}
```